

POLYTECH CLERMONT / EUPI

Stage d'Intégration et de Tests Terrain pour le Challenge MOBILEX

POLYTECH CLERMONT : DÉPARTEMENT S3ER / EUPI : MASTER PAR

RAPPORT DE STAGE DE FIN D'ÉTUDES : ANNÉE ACADÉMIQUE 2025-2026



Élève :

Leonel MBONENG

Dates de stage :

Du 16/03/2026 au 21/08/2026

Entreprise d'accueil :

SHERPA Engineering

Enseignant Référent :

Romuald AUFRERE

Tuteur de Stage :

Thomas BARBIER

Résumé

Ce rapport vise à présenter le travail réalisé durant mon stage au sein du projet MOBITER, mené par SHERPA Engineering, qui a pour but de développer de nouvelles techniques de navigation permettant à un véhicule de se déplacer de manière autonome dans un environnement inconnu, déstructuré, et dont la fiabilité des capteurs n'est pas garantie.

L'approche développée repose sur un modèle numérique d'élévation créé à partir d'une fusion d'informations provenant d'un LIDAR 3D 64 nappes, d'un GNSS, d'un ultrasons et d'une sémantique caméra obtenue grâce à un réseau de neurones Yolo Efficient SAM, avec possibilité de mise à jour avec une méthode SLAM, une planification globale A* et une planification locale prédictive DWA.

Les résultats actuels sont que le robot est capable de naviguer à basse vitesse (2m/s) et de cartographier en toute autonomie dans un environnement déstructuré pouvant contenir des obstacles positifs et négatifs.

Mots-clés

Navigation autonome en milieu déstructuré - Modèle Numérique d'Élévation -
Cartographie SLAM - Yolo World - Efficient SAM - Planification globale A* -
Planification locale DWA - ROS

Abstract

This report aims to present the work carried out during my internship within the MOBITER project, directed by SHERPA Engineering, which aims to develop new robotics techniques to enable a vehicle to move autonomously in an unknown, unstructured environment where sensor reliability is not guaranteed.

The developed approach is based on a digital elevation model created from a fusion of LIDAR, GPS, and ultrasound information, and camera semantics obtained using a Yolo World Efficient SAM neural network, with the possibility of updating using a SLAM method, A* global planning, and DWA predictive local planning.

The current results show that the robot is able to navigate at low speed (2 m/s) and map autonomously in an unstructured environment that may contain positive and negative obstacles.

Keywords

Autonomous navigation in unstructured environment - Digital Elevation Model (DEM) - Mapping SLAM - Yolo World - Efficient SAM - A* global planning - DWA predictive local planning - ROS

Remerciements

En premier lieu, je souhaiterais exprimer ma gratitude envers mon tuteur de stage, Thomas Barbier, ainsi que notre référent technique sur le projet Xavier Dauplain, et Richard Arnaud de l'équipe de Nantes qui m'ont apporté un encadrement de qualité. J'ai beaucoup apprécié les longues discussions où nous prenions le temps de poser et d'approfondir un sujet à travers nos idées. Ils ont su me faire confiance en me donnant beaucoup de liberté et d'autonomie. Étant donné leurs grandes connaissances et compétences en robotique mobile, j'ai eu la chance de pouvoir apprendre beaucoup d'eux tout au long de mon séjour chez Sherpa Engineering.

Ensuite, j'aimerais remercier l'ensemble de l'équipe UPercEa. J'ai sincèrement adoré travailler à vos côtés, de par la bonne ambiance qui règne au sein de l'équipe et par l'entraide que vous vous apportez.

J'aimerais également remercier Hubert Villeneuve de l'INRAE avec qui j'ai réalisé l'ensemble de mes tests sur le terrain. Il m'a donné beaucoup de conseils pour effectuer les tests de manière efficace.

D'autre part, je tiens à remercier Roland Chapuis, mon référent Polytech, pour son aide lors de points théoriques bloquants et pour le temps qu'il m'a accordé sur toutes sortes de sujets.

Enfin, j'aimerais remercier Myriam Doghmi qui m'a donné des conseils dans la rédaction de ce rapport.

Sommaire

Table des figures

Tous les mots suivis d'un astérisque sont définis dans le lexique.

Introduction

Le marché des véhicules autonomes est en plein essor. Les progrès technologiques des dernières décennies ont permis à ce qui s'apparentait autrefois comme un concept de science-fiction de devenir une réalité tangible. En alliant capteurs sophistiqués, systèmes embarqués pouvant traiter un grand nombre de données en temps réel et intégrant plus récemment de l'intelligence artificielle, certaines voitures sont aujourd'hui capables de rouler en toute autonomie dans diverses situations de circulation nécessitant des prises de décisions.

Si ces technologies ont été principalement développées pour des environnements structurés tels que les routes et autoroutes, d'autres secteurs souhaiteraient disposer de technologies similaires pour automatiser ou assister la conduite de véhicules dans des environnements complexes (terrains non structurés), pouvant être évolutifs, plus ou moins couverts par les réseaux de communication et plus ou moins connus. On pourrait par exemple penser à l'agriculture qui pourrait se doter de ces véhicules pour automatiser l'exploitation des cultures ou encore aux domaines à risques où une assistance de conduite avancée pourrait permettre de décharger les pilotes et de libérer des ressources physiques et cognitives, comme par exemple sur un chantier, sur un incendie ou dans une mission militaire.

C'est dans ce contexte que l'Agence de l'Innovation de Défense (AID), l'Agence Nationale pour la Recherche (ANR) et la Direction Générale de l'Armement (DGA), en partenariat avec l'Agence d'Innovation pour les Transports (AIT) et le Centre National d'Etude Spatiale (CNES), ont mis en place le Challenge MOBILEX, un challenge composé de plusieurs séries d'épreuves de difficultés croissantes se déroulant sur trois années, de fin 2023 à fin 2026, mettant en compétition plusieurs équipes composées d'acteurs de la recherche académique ainsi que d'acteurs de l'industrie avec pour objectif principal d'accélérer la recherche et la montée en maturité technologique sur la thématique de la mobilité autonome des véhicules en environnement complexe*.

En tant que SSIS travaillant depuis le début des années 2000 avec d'importants acteurs de l'automobile et ayant activement participé au développement de systèmes ADAS* et à l'automatisation de véhicules, SHERPA Engineering, en partenariat avec ses collaborateurs académiques INRAE et Institut Pascal ont répondu ensemble en tant que consortium MOBITER au Challenge MOBILEX

Cette participation est une excellente occasion pour tous les acteurs de présenter leur savoir-faire, pour les laboratoires d'assembler et de tester leurs travaux de recherche à la pointe de l'état de l'art sur un projet complexe et pour SHERPA Engineering de développer de nouvelles briques logicielles dans un projet de recherche et développement avec une partie financée par l'ANR et l'autre directement par SHERPA Engineering sous forme d'investissement recherche.

Nous verrons dans un premier temps une présentation de SHERPA Engineering, puis nous poursuivrons sur une présentation des attendus du défi en détail ainsi que la répartition des tâches au sein du consortium et le matériel installé sur le robot. Suite à cela, nous nous attarderons sur les briques logicielles développées pour répondre aux attentes de la DGA à l'horizon de la fin de la période du défi 2 et dont la fine compréhension fût essentielle et enfin, nous terminerons par une présentation du travail réalisé au cours du stage.

I SHERPA Engineering

SHERPA Engineering est une société française spécialisée dans la modélisation et la simulation de systèmes complexes pour divers secteurs industriels, notamment l'automobile, l'aéronautique, l'énergie et le ferroviaire. Fondée en 1997, elle se concentre sur le développement de solutions d'ingénierie visant à améliorer la conception, l'optimisation et la validation des systèmes embarqués et des systèmes multiphysiques.

Forte d'un chiffre d'affaires de près de 12 M€ pour environ 90 salariés en France, elle diversifie aujourd'hui ses activités dans le domaine agricole, spatial et de la défense. C'est une entreprise à forte valeur d'innovation qui porte de nombreux projets de recherche et développement et entretient de nombreux partenariats avec des centres de recherche, tels que l'Institut Pascal, l'INRAE ou encore l'Université Gustave Eiffel.

Aujourd'hui, SHERPA Engineering dont le siège est basé à Nanterre dispose de plusieurs sites en France et compte trois filiales à l'étranger : Maroc, Tunisie et Roumanie comme le montre la carte (Figure 1) ci-dessous :

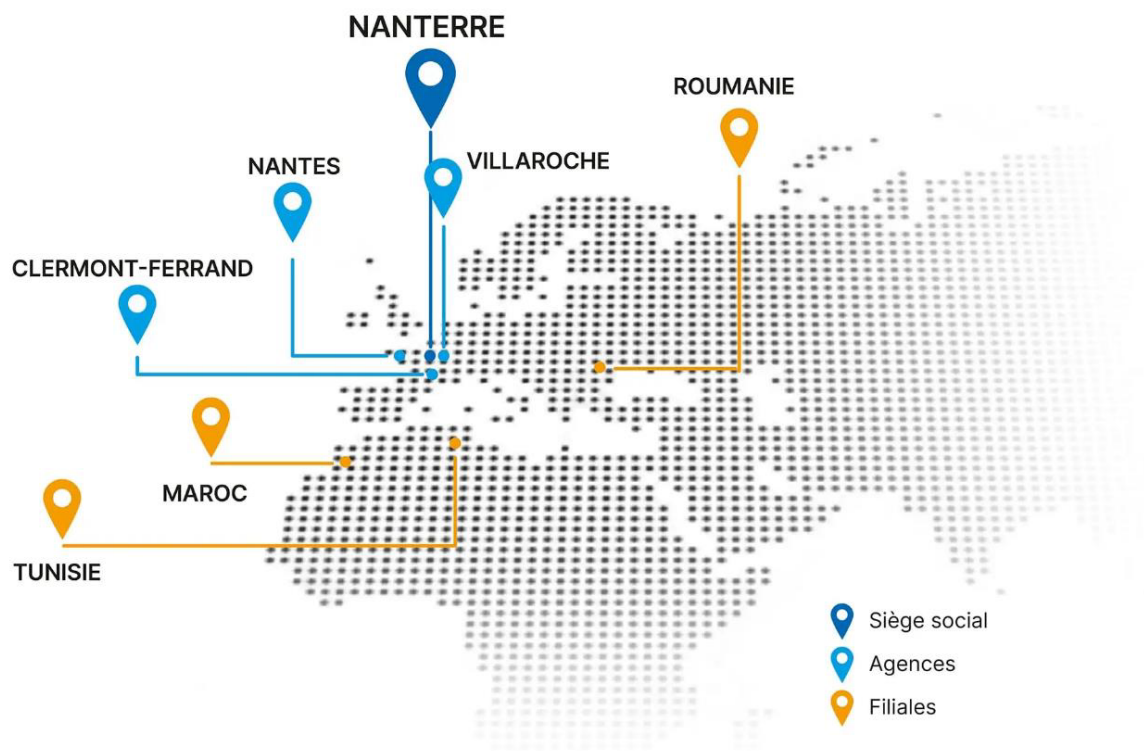


FIGURE 1 – Carte des agences de SHERPA Engineering dans le monde

Elle est divisée en 3 Business Units (BU) :

- BU Décarbonation : Spécialité Energies
- BU MBD : Spécialité Modélisation et Système

- BU AS : Spécialité Aéronautique

Voici l'organigramme (Figure 2) de l'entreprise afin de mieux comprendre la structure.

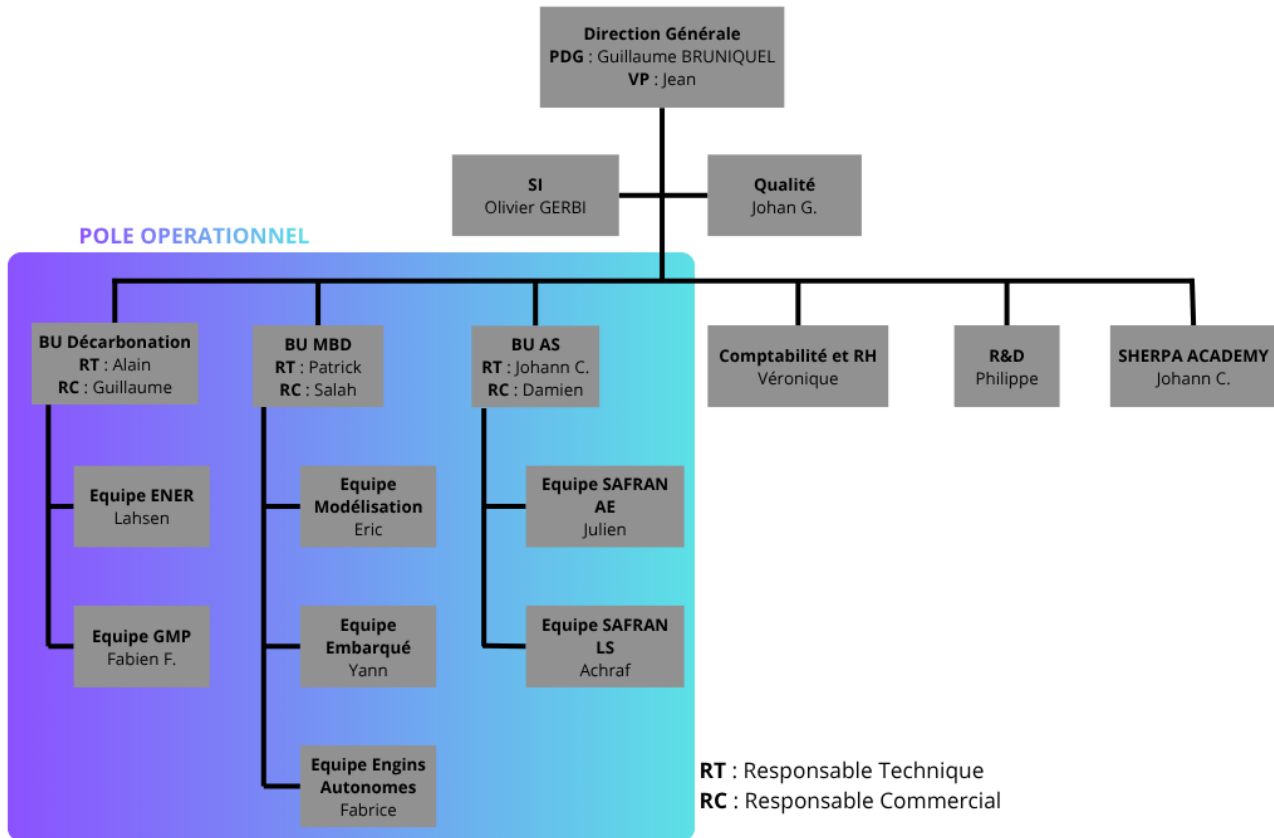


FIGURE 2 – Organigramme de SHERPA Engineering

Au coeur de cette organisation, je fais partie de l'équipe Engins Autonomes, aussi appelée UPerCEa, dirigée par Fabrice Peyrin, au sein de la Business Unit MBD. Cette équipe est partagée entre l'agence de Clermont-Ferrand et celle de Nantes. Mon tuteur de stage est Thomas Barbier, ingénieur en robotique spécialisé en vision artificielle.

II Présentation du Challenge MOBILEX

Comme mentionné précédemment, le Challenge MOBILEX* vise à accélérer la recherche et à faire mûrir technologiquement le domaine de la navigation autonome des véhicules en Environnement complexe*.

Pour ce faire, les organisateurs ont lancé un appel à projet auquel pouvaient candidater des consortiums composés d'au moins un acteur industriel et d'un acteur de la recherche française. L'objectif était de rendre un véhicule imposé autonome pour des cas de situations donnés et de proposer des fonctionnalités/aides facilitants le contrôle par un téléopérateur.

À l'issue de l'appel à projets, sept consortiums ont été retenus comme ci-après (Figure 3) :

PANAME	 ONERA THE FRENCH AEROSPACE LAB	 Génération ROBOTS	
REFLEX	 cea	 magellium	
NAVIR	 HAWAI .tech	 Inria	
MOBITER	 SHERPA engineering	 INRAE	 INSTITUT PASCAL institut de l'énergie et des systèmes
TAURUS	 ENSTA	 INSTITUT POLYTECHNIQUE DE PARIS	 exail
MEEDIAN	 SAFRAN AEROSPACE DEFENCE SECURITY	 MINES PARIS	 PSL
ASTRA	 MANITOU GROUP	 ESIGELEC INGÉNIEUR.E.S GÉNÉRALISTES SYSTÈMES INTELLIGENTS ET CONNECTÉS	

FIGURE 3 – Liste des équipes du challenge

1 Les attendus du challenge

1.1 Organisation générale

Le Challenge Mobilex est organisé par les acteurs suivant (Figure 4) :



FIGURE 4 – Organisateur du challenge Mobilex

Il se déroule sur trois années et est composé de trois défis de difficulté croissante, à savoir un défi par année. Les modalités de ces défis ne sont communiquées aux équipes qu'au début de chaque défi [?]. À la fin de chaque année, toutes les équipes sont convoquées sur un site de la DGA pour y être évaluées sur leurs propositions en réponse aux situations rencontrées lors du défi.

L'organisation de chaque défi se déroule comme suit (Figure 5) :

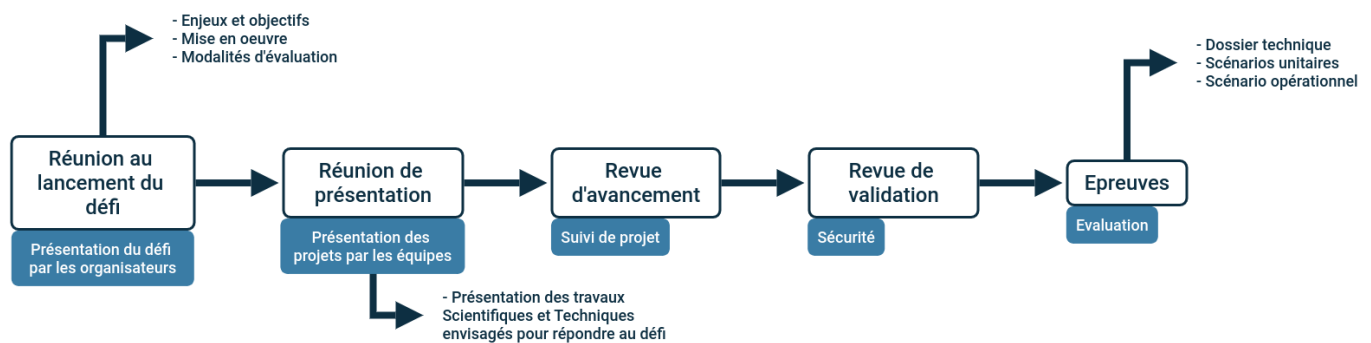


FIGURE 5 – Déroulement d'un défi

1.2 Fonctions attendues

Pour chaque défi, une liste de nouvelles fonctions attendues est donnée. Les fonctions demandées lors des précédents défis doivent également fonctionner lors des scénarios du défi en cours (sauf indication contraire de la DGA). Ces fonctions peuvent être classées selon deux modes de fonctionnement : le mode pilote automatique et le mode téléopération assistée.

1.2.1 Mode pilote automatique

Pour le premier défi, les fonctions attendues étaient les suivantes :

- Détection et gestion locale des obstacles positifs franchissables et non franchissables.
- Détection et gestion locale des bas-côtés positifs.
- Fonction rejeu de trajectoire.
- Fonction retour sur trace.
- Évitement de zones d'exclusion, d'interdiction et passage par des points de passage imposés.

Le second défi a quant à lui demandé l'ajout des fonctions suivantes :

- Détection et gestion locale des obstacles négatifs franchissables et non franchissables.

- Détection et gestion locale des obstacles transparents/réfléchissants.
- Détection et gestion locale des bas-côtés négatifs.
- Prise en compte des pentes positives et négatives.
- Détection et gestion locale des dévers.
- Gestion des situations dégradées (pannes capteurs, perte de communication avec le poste de téléopération, ...).

Lors du troisième et dernier défi, les équipes se sont vu supprimées certaines fonctions imposées au cours des précédents défis, à savoir :

- Le rejeu de trajectoire et le retour sur trace.
- Détection et gestion locale des obstacles transparents/réfléchissants.
- Détection et gestion locale des obstacles à bascules.
- Le changement artificiel de gabarit.
- Gestion des pannes capteurs.

Et se sont vu rajoutées les nouvelles fonctions suivantes :

- Gestion des pentes et dévers avec obstacles franchissables ou non.
- Gestion de la végétation et des flaques d'eau.
- Gestion des situations dégradées (perte des données gnss, perte de communication avec le poste de téléopération, ...).

1.2.2 Mode téléopération assistée

Pour le premier défi, une IHM fonctionnant sur un PC déporté a été demandée pour, au minimum, pouvoir choisir le mode de pilotage, renvoyer les commandes réalisées par le téléopérateur lors d'un pilotage manuel et fournir un retour vidéo annoté. Ce retour vidéo doit contenir des informations concernant la compréhension de l'environnement établie par la brique logicielle de l'équipe ainsi que les indications de direction pour gérer la trajectoire.

Lors du défi 2, un sous-mode *Safety First* en téléopération assistée a été demandé. Ce mode vise à préserver l'intégrité du robot en imposant le respect des contraintes de navigation données par les organisateurs, en reprenant temporairement le contrôle du véhicule en cas de danger. ce sous-mode doit être activable ou désactivable à tout moment depuis l'IHM.

1.3 Contraintes de navigation à intégrer lors du développement

1.3.1 Détection et gestion des obstacles

Tout obstacle d'une hauteur de plus de 30 cm (positive ou négative) par rapport au sol est considéré comme non franchissable et doit être évité. Le franchissement d'obstacles entre 5 et 30 cm doit être fait à une vitesse réduite inférieure à 5 km/h et avec arrêt préalable pour les obstacles entre 15 et 30 cm.

La Figure 6 est un tableau récapitulatif sur la franchissabilité des obstacles en fonction de leur hauteur et de la vitesse du robot :

	avec arrêt	< 5 km/h	> 5 km/h
< 5 cm	Franchissable	Franchissable	Franchissable
entre 5 cm et 15 cm	Franchissable	Franchissable	Non franchissable
entre 15 cm et 30 cm	Franchissable	Non franchissable	Non franchissable
> 30 cm	Non franchissable	Non franchissable	Non franchissable

FIGURE 6 – Gestion des obstacles en fonction de leur hauteur/profondeur et de la vitesse du robot

1.3.2 Détection et gestion des bas-côtés

Les bas-côtés (haies, murs, fossés, barrières, etc.) d'une hauteur de plus de 30 cm (positive ou négative) sont détectés à tout instant, quel que soit le mode de pilotage. Le suivi d'un bas-côté doit se faire à une distance fixe paramétrable de manière autonome.

1.3.3 Détection et gestion des pentes et dévers avec obstacles

Les pentes et dévers de plus de 20° sont considérées comme non franchissables et sont détectées à tout instant, quel que soit le mode de pilotage. Les pentes correspondent à des inclinaisons du sol dans le sens de la trajectoire du robot, les roues à l'avant ont donc une altitude différente de celles à l'arrière. Les dévers quant'à eux correspondent à des inclinaisons transversales du sol par rapport au sens de la trajectoire du robot. Les roues d'un côté du robot ont donc une altitude différente de celles du côté opposé. Le franchissement des pentes et dévers avec obstacles est régit comme suit (Figure 7) :

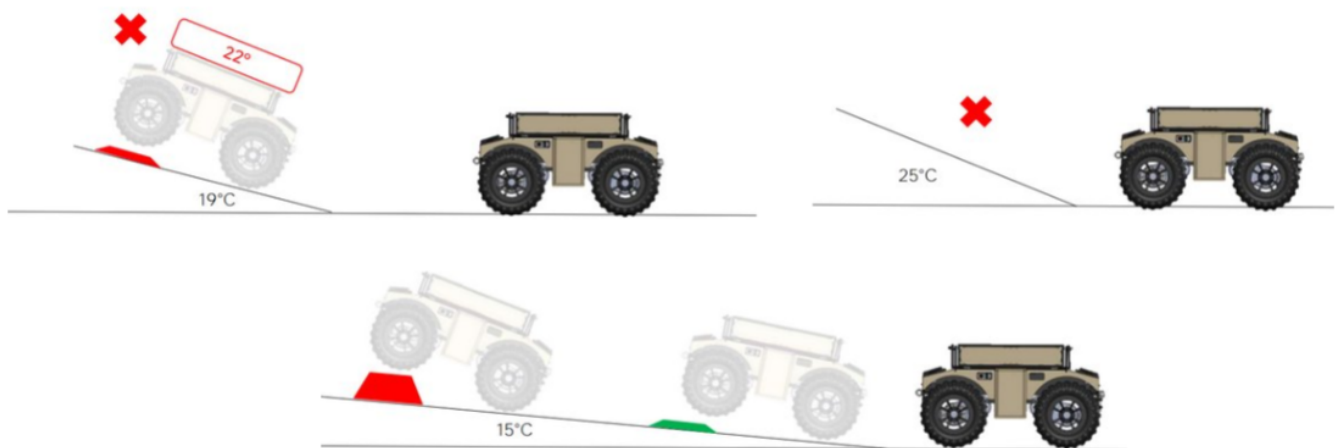


FIGURE 7 – Gestion des pentes avec obstacles

1.3.4 Gestion des situations dégradées

Lors du défi 2, la Brique MOBILEX* se devait d'être robuste à des conditions environnementales dégradées telles que la pluie, le brouillard, la fumée, par contre au cour du défi 3 il est attendu quelle soit robuste à une couverture GNSS dégradée et aux pertes de communication avec la station de contrôle débarquée. Ces dégradations peuvent être naturelles ou provoquées lors des épreuves. Lorsqu'une ou plusieurs de ces conditions interviennent, les fonctions doivent toujours pouvoir être réalisées.

1.3.5 Gestion des pannes capteurs

La Brique MOBILEX* doit être robuste aux pannes capteurs individuelles brèves (inférieures à 10sec) et prolongées (inférieures à 2min) des capteurs suivants :

- GPS
- Caméra
- LIDAR*

Au cours du défi 2, ces pannes devaient pouvoir être simulées en coupant momentanément les capteurs souhaités par contre cette fonction n'est plus évaluée lors du défi 3. En cas de panne, les fonctions doivent toujours pouvoir être réalisées.

1.3.6 Points de passage imposés

L'IHM doit permettre au téléopérateur de renseigner des points de passage imposés par les organisateurs. Le robot doit obligatoirement passer par chacun des points de passage renseignés en respectant l'ordre imposé. Ces points de passage sont définis par leurs coordonnées géographiques, en unités cartésiennes par projection UTM 31 dans le système géodésique WGS 84 avec une précision décimétrique.

1.3.7 Rejeu de trajectoire

La Brique MOBILEX* doit permettre de réaliser des rejeux de trajectoire à tout moment et quel que soit le mode de pilotage courant. Elle doit donc être capable d'enregistrer une trajectoire à tout moment. Le rejeu de trajectoire se fait de manière autonome après repositionnement du robot au début de cette dernière et doit prendre en compte la potentielle apparition de nouveaux obstacles, ou l'altération d'obstacles déjà existants.

Une option doit permettre d'optimiser automatiquement une trajectoire parcourue au préalable en réduisant le temps de réalisation de cette dernière tout en conservant le ralliement des points de passage dans le même ordre. La durée du processus d'optimisation est comptabilisée dans le temps de réalisation du rejeu.

1.3.8 Retour sur traces

La Brique MOBILEX* doit permettre de rejoindre la position de départ du robot en suivant en sens inverse l'intégralité de la trajectoire préalablement parcourue à tout moment, quel que soit le mode de pilotage courant. Le retour sur trace se fait de manière autonome et doit prendre en compte la potentielle apparition de nouveaux obstacles, ou l'altération d'obstacles déjà existants.

L'objectif à rejoindre par défaut est la position du robot au début de l'épreuve. Une nouvelle position à rejoindre peut être imposée par les organisateurs. Il s'agira alors d'un point appartenant à la trajectoire à l'aller.

Le retour sur traces peut relever d'un contexte d'urgence et peut altérer la limitation de vitesse. Il doit pouvoir se faire en marche arrière et en marche avant, avec la possibilité de changer de sens en cours de retour.

1.3.9 Zones d'exclusion

Les zones d'exclusion sont des polygones définis par les coordonnées géographiques (UTM) des points qui les composent et peuvent ne pas être matérialisées sur le terrain. Le robot ne doit pas pénétrer dans les zones d'exclusion sous peine de pénalité. Une zone qui serait prise en compte comme zone d'exclusion alors qu'elle n'en est pas une entraînera également une pénalité.

1.3.10 Zones d'interdiction

Les zones d'interdiction correspondent à des contraintes de sécurité. Ce sont des polygones définis par les coordonnées géographiques (UTM) des points qui les composent et peuvent ne pas être matérialisées sur le terrain. Leur chevauchement entraîne un arrêt d'urgence ainsi qu'une pénalité.

1.4 Évaluation des développements pour le défi en cours.

L'évaluation des développements se fait à chaque défi et comporte trois notes : une note liée à la revue de validation, une note liée à la soumission du dossier scientifique et technique et une note liée aux épreuves. Elles visent à démontrer les capacités de développement de l'équipe et les performances du système développé.

Lors de la journée d'épreuves sur un site de la DGA, les robots des équipes doivent accomplir une suite de scénarios donnés dans le règlement particulier du défi permettant d'évaluer des fonctions seules ou un ensemble de fonctions. Les équipes sont notées sur la bonne prise en compte de l'ensemble des contraintes fournies par les organisateurs et le bon fonctionnement des fonctions avec des critères de précision et/ou de rapidité.

2 Le consortium MOBITER

Pour répondre au mieux aux ambitions du Challenge MOBILEX*, SHERPA Engineering s'est associée avec ses partenaires académiques INRAE et Institut Pascal, avec lesquels elle collabore régulièrement grâce à une proximité géographique, un laboratoire qui était jusqu'à récemment commun pour SHERPA Engineering et l'Institut Pascal, et des propositions régulières de thèses CIFRE.

Cette proximité permet de répartir aisément les tâches à réaliser en fonction de l'expertise de chacun et de faciliter la communication entre les différents acteurs, notamment grâce à un canal Teams sur lequel sont partagés les avancées et différents documents et où une réunion hebdomadaire a lieu, permettant de présenter le travail de chacun et de planifier les tests sur le terrain. Les apports de chaque partie sont les suivants :

2.1 SHERPA Engineering

SHERPA Engineering, coordinatrice du projet, réalise, sur la base d'un existant, la mise en place de la base hardware du projet en l'adaptant pour répondre aux exigences du challenge. Elle réalise les tâches d'ingénierie du projet et a la charge d'une partie de l'algorithmie liée à la perception et à la planification.

2.2 INRAE

L'INRAE apporte son expérience dans la robotique agricole, notamment sur les nouvelles lois de commande, et est responsable de l'expérimentation grâce à l'utilisation de ses sites à Aubière (Puy-de-Dôme) et Montoldre (Allier).

2.3 Institut Pascal

L'Institut Pascal fournit des conseils sur le développement scientifique (fusion de données, traversabilité, gestion des risques, planification) qui sont mis en application par une ressource interne.

3 Présentation du robot Barakuda et du matériel installé

3.1 Robot BARAKUDA

Pour répondre au challenge, la DGA a mis à disposition de chaque équipe un robot BARAKUDA de la société SHARK ROBOTICS (Figure 8).



FIGURE 8 – Robot BARAKUDA

Ce robot est une plateforme mobile pilotable jusqu'à 500 mètres de distance en terrain dégagé avec la télécommande fournie, doté de quatre roues motorisées non directionnelles pour une masse de 500 kg. Il est capable d'atteindre une vitesse de 20 km/h avec une charge utile pouvant aller jusqu'à 500 kg, de rouler sur des pentes allant jusqu'à 40°, des dévers allant jusqu'à 35° et de franchir des obstacles allant jusqu'à 30 cm. Il peut atteindre jusqu'à 10 heures d'autonomie en situation opérationnelle.

La communication entre le robot et la Brique MOBILEX* se fait via des trames UDP (protocole réseau utilisé pour des applications sensibles au temps) permettant d'envoyer des commandes de vitesse linéaires et angulaires et de récupérer des informations sur l'état du

robot (vitesses et puissances des roues, inclinaisons, charge des batteries et température) par instruction de type GET.

3.2 Matériel ajouté par le consortium

Les équipes n'ont pas l'autorisation de modifier le robot en lui-même mais sont libres d'ajouter leur propre matériel tant qu'il respecte les dimensions à ne pas dépasser pour le challenge et la charge utile du robot. Nous avons donc choisi d'ajouter le matériel suivant (Figure 9) :

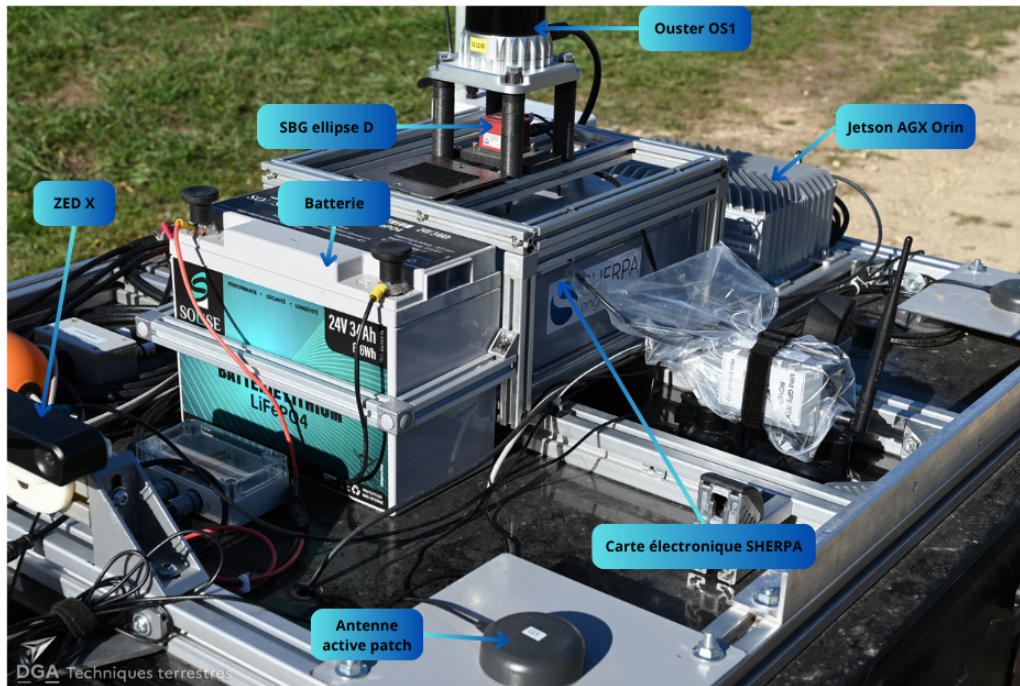


FIGURE 9 – Plateau capteurs hardware

3.2.1 Calculateur Syslogic RML A4AGX

Lors du premier défi, SHERPA Engineering avait utilisé un calculateur Syslogic RML A3 qu'elle possédait déjà mais s'est rendu compte qu'il y aurait un problème de performances lors de l'ajout de nouvelles briques pour répondre aux défis ultérieurs. Elle a donc opté pour le modèle supérieur, le RML A4AGX.

Ce calculateur est un PC industriel IP67 à dissipation thermique passive, basé sur un System On Module Nvidia Jetson AGX Orin. La puce Jetson AGX Orin intègre notamment un processeur ARM Cortex A78AE de 12 cœurs de 2,2 GHz, un GPU Nvidia Ampere de 2048 cœurs CUDA, 64 Go de RAM LPDDR5, et des accélérateurs d'inférences Nvidia v2. Il dispose également de 6 contrôleurs Gigabits Ethernet indépendants et deux contrôleurs USB 3.1. Un disque dur SSD de 2 To interne en PCIe a été ajouté pour enregistrer des logs.

Ce choix a été motivé par l'excellente performance énergétique des puces Nvidia Jetson et la présence d'un GPU intégré, ce qui favorise leur intégration dans des applications robotiques et embarquées exigeantes.

3.2.2 INS SBG Ellipse-D

Ce système de navigation inertielle, placé au centre du plateau, permet d'estimer la pose (altitude et position) et les vitesses linéaires et angulaires du robot. Ce capteur intègre une centrale inertielle calibrée et compensée en température, et un récepteur GNSS double antenne. La fusion INS se fait par filtrage de Kalman en interne.

Le choix de ce capteur était motivé par la volonté d'intégrer une solution INS qualitative dans un format compact et robuste avec un coût maîtrisé.

3.2.3 LIDAR 3D Ouster OS1

Ce LIDAR* 3D 64 nappes, placé au-dessus de l'INS, permet d'observer l'environnement à 360° autour du robot.

Le choix de ce capteur était motivé par le paradigme choisi pour la perception. La compacité, la résilience affichée et le coût des LIDAR*s Ouster ont favorisé l'intégration de ce modèle particulier.

3.2.4 Caméra ZED X

Cette caméra, placée à l'avant du robot, est une caméra stéréo permettant, en plus d'obtenir un retour visuel, de fournir un nuage de points 3D dense à l'avant du robot.

Le choix de ce capteur était motivé par la fusion du nuage de points avec celui du LIDAR* pour combler d'éventuels manques d'informations dans la carte et d'obtenir un retour vidéo de bonne qualité pour la téléopération ainsi que pour de la vision artificielle.

3.2.5 Capteurs Ultrasoniques 200kHz

Ces capteurs, un placé à l'avant du robot, permettent de compléter la perception de l'environnement lors de l'apparition d'obstacles complexes à gérer pour un LIDAR*.

3.2.6 Sous-système électronique SHERPA

Ce sous-système a été conçu antérieurement par SHERPA et est composé d'un ensemble de cartes électroniques dans un boîtier IP67 usiné sur mesure, qui permettent d'assurer la synchronisation, le pilotage commandé et le multiplexage de données capteurs. Le boîtier intègre une centrale inertielle XSens MTi-7 ainsi qu'un récepteur GNSS F9P de Ublox, permettant une redondance capteur avec la SBG Ellipse-D.

3.2.7 Antennes 5 GHz et 868 MHz

Ces antennes permettent d'établir la communication entre le PC débarqué et le PC embarqué. Il y a donc une antenne 5 GHz et 868 MHz sur la station débarquée et sur le robot. Par contre seule l'antenne 5 GHz du robot est utilisée pour la communication avec le PC débarqué car elle offre une meilleure bande passante et une plus grande robustesse aux obstacles (végétation, bâtiments, ...).

3.2.8 Poste de Contrôle Commande (PCC) ou PC débarqué

Ce Poste constitué d'un PC portable, d'antennes omni (5GHz et 868MHz), d'une manette de jeu, d'une batterie + onduleur permet au téléopérateur de contrôler le robot à distance et de recevoir les retours vidéo et d'informations sur l'état du robot. Le PCC se présente comme suit (Figure 10) :



FIGURE 10 – PC débarqué ou Poste de Contrôle Commande (PCC)

On obtient alors sur le robot l'architecture hardware (Figure 11) :

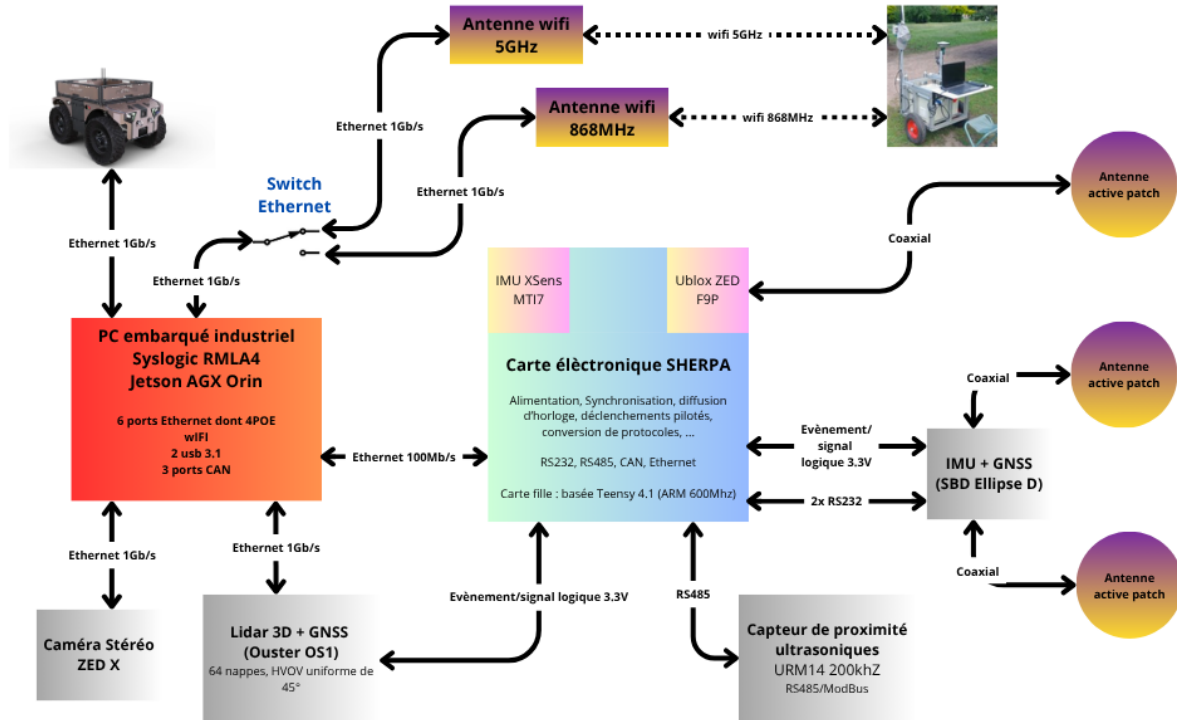


FIGURE 11 – Diagramme synoptique de l'architecture hardware du système

4 Présentation du simulateur Gazebo

Bien que tous les membres du consortium aient une présence à Aubière, une partie de l'équipe de développement de SHERPA Engineering se situe dans l'agence de Nantes, les empêchant de réaliser des tests directement sur le robot. Pour palier ce problème, SHERPA Engineering a mis en place un simulateur Gazebo (Figure 12) reprenant le plus fidèlement possible les données à fournir et les données renvoyées par le robot et ses capteurs (cf. RQT graph en annexe [I]) pour pouvoir réaliser une première phase de tests visant à déceler de potentiels problèmes sur les briques de haut niveau (perception, planification et calcul des commandes). Le simulateur contient divers obstacles correspondant aux cas de situations donnés par la DGA lors de la présentation des modalités du défi en début d'année et est enrichi au fur et à mesure des défis.



FIGURE 12 – Simulateur Gazebo

III Présentation des briques logicielles du robot

Les développements logiciels se font sous le middleware ROS* (noetic), ce qui permet de développer sur une base standardisée offrant de nombreuses fonctionnalités et outils pour faciliter l'intégration des apports des différents acteurs à la Brique MOBILEX*. L'utilisation de ce middleware nous permet d'organiser le code comme un ensemble de briques élémentaires effectuant des fonctions bien définies prenant des données en entrées et retournant les résultats en sortie. L'assemblage de ces différentes briques constitue notre Brique MOBILEX*. Cette organisation permet si nécessaire d'ajouter, retirer ou remplacer des fonctions élémentaires sans avoir à modifier le reste des fonctions. De plus, les fonctions intensives en calcul sont développées en C++ pour répondre à des contraintes de temps réel et les autres en Python car plus rapide à développer.

Un schéma synoptique de l'architecture du système est donné ci-dessous (Figure 13) :

- Les modules et données de bas-niveau et hardware sont représentés en rouge.
- Les modules et données de perception/ localisation sont représentés en vert.
- Les modules et données de navigation sont représentés en bleu.
- Les modules et données de contrôle sont représentés en jaune.
- Les modules et données d'interface, de logique, de contrôle de mission ou de visualisation sont représentés en blanc.

Il est à noter que l'ensemble des fonctions publient également des diagnostics, non représentés ici par souci de lisibilité.

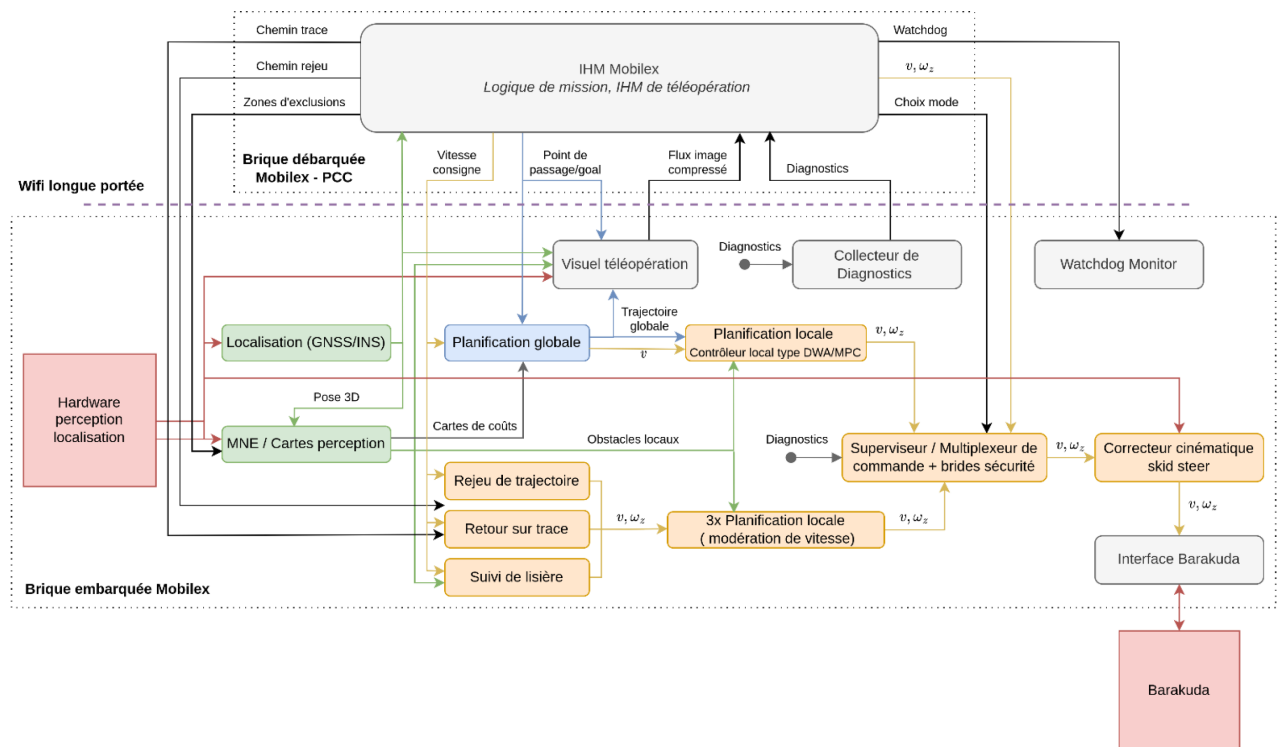


FIGURE 13 – Diagramme synoptique de l'architecture logicielle du système

1 Briques de perception de l'environnement

Les briques de perception de l'environnement ont pour objectif de récolter des données provenant de capteurs majoritairement extéroceptifs pour pouvoir en concevoir une modélisation.

1.1 Modèle Numérique d'Élévation

La brique de cartographie selon un MNE se compose de deux parties : une librairie en C++ permettant de générer le MNE et un noeud ROS* permettant d'utiliser le MNE pour concevoir des cartes d'élévations correspondant aux modalités du Challenge.

Le MNE permet de concevoir une représentation en vue de dessus de l'élévation de l'environnement sous la forme d'une carte de chaleur discrète (Figure 14) d'une résolution de 10x10cm. Il est construit à partir des données de position du robot, de ses vitesses 3D et des points LIDAR*. Ces données sont reçues de façon asynchrone et mises dans des tampons. Pour chaque nouvelle donnée, une partie de la carte, de 40x40m centrée autour de la position du robot, est publiée. Le MNE est mis à jour sur réception d'un scan LIDAR*, provoquant un ensemble d'interpolations pour trouver la vitesse et la position exacte du robot au moment exact du début du scan. Le nuage de points fourni par le LIDAR* est ainsi corrigé en distorsion en utilisant le début du scan comme référence et replacé dans le repère monde. Chaque point du nuage ainsi constitué sert alors à actualiser les valeurs d'élévations maximales présentes dans les tuiles de la carte grâce à un filtrage de Kalman* se servant de la position du robot et de la position des impacts LIDAR* et de leurs covariances associées.

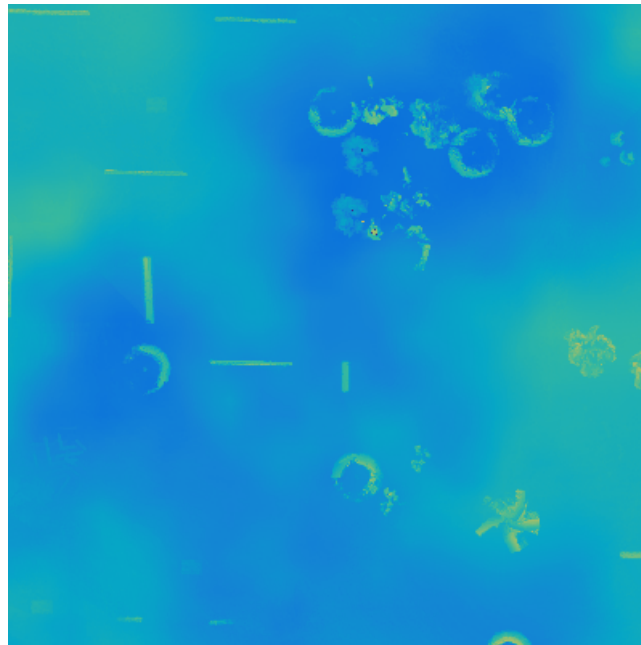


FIGURE 14 – Extrait du Modèle Numérique d'Élévation sous forme d'une carte de chaleur

Le noeud ROS* associé au MNE récupère la carte de 40x40m centrée autour du robot pour construire une carte de gradients, en utilisant un filtre de Sobel de taille 3x3 tuiles (30x30cm). Le Filtre de Sobel est un opérateur utilisé en traitement d'image pour faire de la détection de contours. Il permet de calculer le gradient d'intensité de chaque pixel ce qui indique la direction de la variation ainsi que son intensité, ce qui permet de faire ressortir les contours d'une image.

En détournant cet usage sur notre carte d'élévation où la couleur nous donne une information sur la hauteur, on obtient une information sur les variations de hauteur pour chaque tuile.

On vient ensuite seuiliser notre carte de gradients afin d'obtenir une information correspondant aux différentes hauteurs d'obstacles à différencier pour le défi. La superposition de ces cartes permet d'obtenir une représentation des obstacles autour du robot comme illustré (Figure 15) :

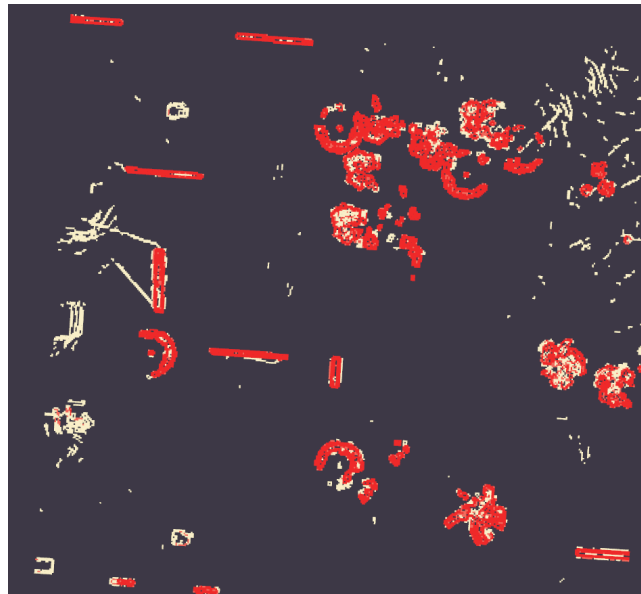


FIGURE 15 – Représentation des obstacles autour du robot

Il est possible de demander une carte de coût associée à cette représentation selon un modèle client/serveur. Cette fonction est exploitée par le planificateur global lorsqu'il estime qu'un recalcul de la trajectoire globale est nécessaire.

Il existe de nombreuses méthodes pour cartographier l'environnement [?], permettant d'obtenir des représentations adaptées à la situation avec les capteurs disponibles [?, ?, ?]. La majorité de ces méthodes reposent néanmoins sur une fusion de données proprioceptives (odométrie, IMU) et extéroceptives (caméra, LIDAR*, ultrason, etc) avec un filtrage Kalman*.

1.2 Segmentation d'image

La brique de segmentation d'image* permet d'extraire des informations sémantiques à partir d'une image caméra. Pour ce faire, elle utilise une combinaison de deux réseaux de neurones : un détecteur open-set, YOLO World [?], et un réseau de segmentation*, Efficient SAM [?].

Le détecteur open-set YOLO World est un réseau de neurones qui prend en entrée une image ainsi qu'un texte décrivant l'objet à détecter et à localiser. Il fournit en sortie des boîtes englobantes représentant les zones où les objets ont été détectés.

Ce détecteur est dit open-set car, contrairement aux autres modèles YOLO, il est capable de détecter des objets qui ne figuraient pas dans son jeu de données d'entraînement initial. Cela est rendu possible grâce à l'intégration de techniques avancées d'apprentissage automatique et

de traitement du langage naturel, permettant au modèle de comprendre et d'interpréter une large gamme d'objets à partir de descriptions textuelles.

Une fois les objets détectés dans l'image, la sortie de YOLO World est utilisée comme entrée pour Efficient SAM, une version optimisée pour l'embarqué du réseau de neurones SAM [?]. Efficient SAM se charge de segmenter précisément les objets en coloriant les pixels au sein des boîtes englobantes, ce qui nous donne le résultat (Figure 16) :



FIGURE 16 – Résultat de la segmentation obtenu avec la combinaison de YOLO World et Efficient SAM

Pour assurer un fonctionnement en temps réel, ces réseaux sont optimisés en supprimant les poids dont la valeur est proche de zéro, réduisant ainsi le nombre de paramètres. Cette optimisation est réalisée avec TensorRT. Une optimisation plus performante également envisagée serait d'utiliser des réseaux de neurones plus précis comme SAM et de réaliser une distillation de connaissances en entraînant un réseau de neurones constitué de moins de paramètres et spécialisé pour notre cas d'application à l'aide des sorties du réseau plus important.

L'objectif à terme est de fusionner cette information sémantique dans la brique du Modèle Numérique d'Environnement (MNE) pour ajouter une notion de traversabilité aux obstacles de la carte.

2 Briques de planification

L'objectif des briques de planification est de calculer une trajectoire à suivre pour remplir un objectif à partir de la perception de l'environnement.

2.1 Planification globale

La brique de planification globale se sert des cartes de coûts générées par la brique du MNE pour recalculer périodiquement un chemin continu reliant le robot à un point dans le repère

monde. Le chemin est obtenu par un algorithme de recherche de chemin classique de type A* [?] modifié.

Cet algorithme est une amélioration de l'algorithme de Dijkstra [?] qui permet de trouver le chemin optimal selon un critère donné dans un graphe.

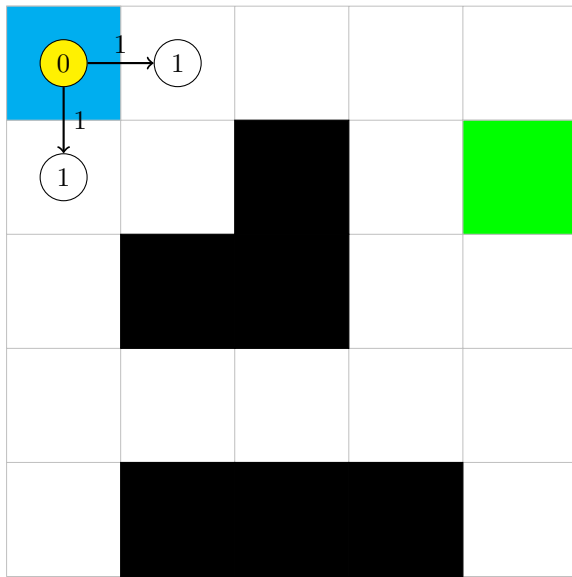
Pour trouver ce chemin, l'algorithme de Dijkstra attribue à son noeud de départ un coût de 0 et attribue aux chemins vers les noeuds voisins des poids, par exemple la distance à vol d'oiseau. Le coût d'un noeud voisin correspond à la somme entre le coût du noeud précédent et le poids du chemin pour rejoindre le nouveau noeud (équation (??)).

$$c(n) = c(k) + p(k, a) \quad (1)$$

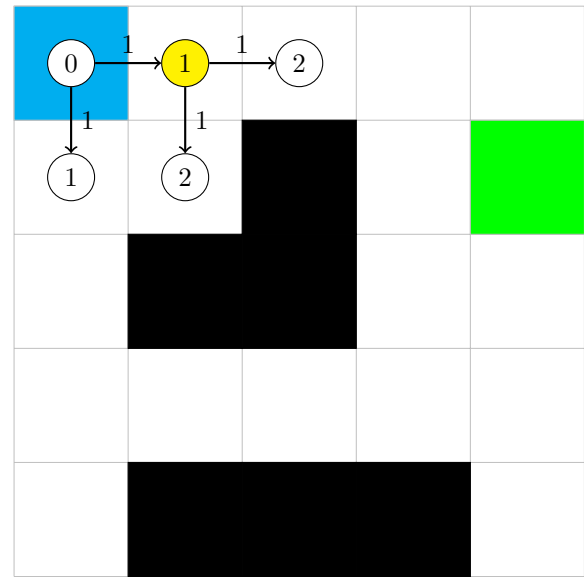
où n est le noeud exploré, k est le noeud précédent et a est l'action de déplacement utilisée depuis le noeud k .

Il explore alors le noeud dont le coût est le plus faible en ajoutant les poids des chemins voisins ainsi que le coût des noeuds voisins et répète l'exploration jusqu'à atteindre l'objectif.

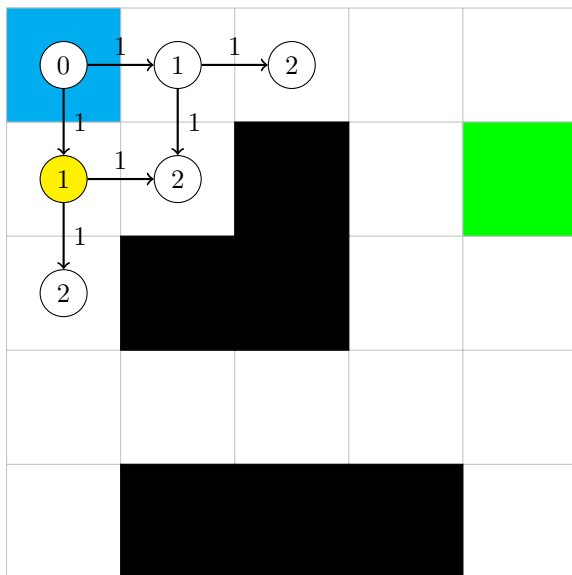
La Figure 17 est une représentation étape par étape de l'exploration d'une carte par l'algorithme de Dijkstra. Seuls les mouvements (haut, bas, gauche, droite) sont admissibles. La case bleue est le point de départ de l'algorithme, la case verte l'objectif à atteindre et le cercle jaune le noeud en cours d'exploration :



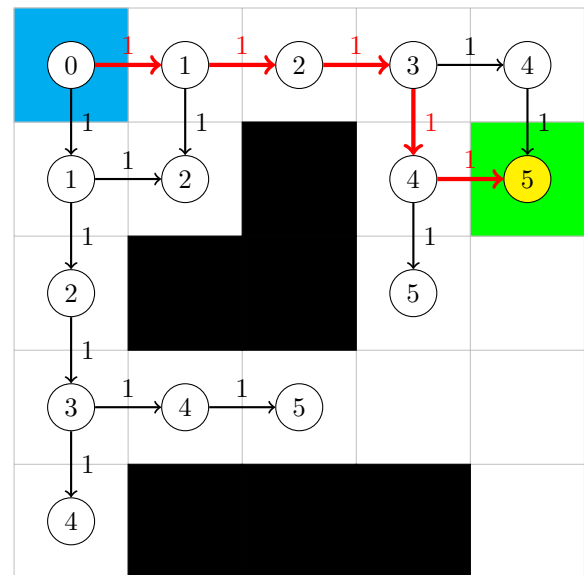
(a) Exploration du noeud de départ



(b) Exploration du noeud de coût le plus faible



(c) Nouvelle exploration



(d) Résultat de l'exploration de la carte

FIGURE 17 – Fonctionnement de l'algorithme de Dijkstra

Face au problème d'exploration aveugle (brute-force) dont fait face l'algorithme de Dijkstra, l'amélioration apportée par l'algorithme A* est d'ajouter un terme appelé l'heuristique dans le coût d'un noeud qui fournit une estimation du coût pour rejoindre l'objectif depuis ce noeud.

Le coût utilisé lors de la sélection du noeud à explorer suit alors l'équation (??) :

$$f(n) = c(n) + h(n) \quad (2)$$

où $c(n)$ est la somme des poids pour atteindre le noeud n (comme dans l'algorithme de Dijkstra) et $h(n)$ est l'heuristique ajoutée pour aiguiller l'exploration vers l'objectif.

Cette modification provoque une exploration plus rapide du graphe dès lors que l'heuristique est correctement choisie. Ce qui entraîne une réduction de la charge computationnelle dans le temps, ainsi qu'une plus faible mobilisation et consommation de ressources.

Dans notre cas, l'heuristique est la distance à vol d'oiseau entre le noeud et l'objectif tandis que le poids entre deux noeuds est une somme entre la distance parcourue, un critère sur la similitude par rapport à la planification précédente sur les premiers mètres du chemin (afin d'éviter des oscillations) et un critère pondéré lié à la carte de coût du MNE. Le niveau de risque acceptable par le robot peut ainsi être choisi en modifiant la pondération de ce dernier coût, une faible pondération impliquant un chemin pouvant plus se rapprocher des obstacles détectés.

La carte de coût utilisée (Figure 18) est similaire à une CostMap de ROS*, avec une zone critique définie autour de chaque obstacle correspondant aux zones où l'on considère que le robot va entrer en collision, une zone de danger où l'on considère que selon l'orientation le robot peut entrer en collision et un champ de potentiel décroissant avec la distance à l'obstacle.



FIGURE 18 – Carte de coût utilisée par la planification globale

Les zones critiques sont par définition impossibles à traverser tandis que les zones de dangers sont fortement pénalisées car il peut y avoir collision selon l'orientation du robot et sont donc souvent considérées uniquement lorsqu'il n'y a pas de solution alternative. Etant donné le manque de prise en compte de l'orientation du robot et l'absence de prise en compte de son modèle d'évolution, les chemins traversant des zones de danger pourraient ne pas être valides au sens de la traversabilité mais la brique de contrôle en aval, la planification locale, qui elle prend en compte l'orientation et l'enveloppe réelle (rectangulaire) du robot peut quand même trouver une solution à partir de ce chemin. Le surcoût computationnel qui interviendrait si

l'on modifiait la carte de coûts en fonction de l'orientation du robot a encouragé à choisir ce compromis, en sachant que seuls les premiers mètres du chemin nous intéressent pour trouver une direction de déplacement privilégiée.

Les algorithmes de Dijkstra et A* fournissent une solution optimale pour la situation donnée : carte échantillonnée et heuristique donnée pour l'A*. Il existe d'autres algorithmes de planification permettant de s'affranchir de la limitation de l'échantillonnage et permettant d'explorer l'environnement plus rapidement en générant des noeuds aléatoirement dans la carte et en essayant de les relier pour former un chemin jusqu'à l'objectif comme la Probabilistic Road-Map [?], le RRT [?] et le RRT* [?]. Les chemins générés ne sont plus optimaux et peuvent ne pas fournir de solution alors qu'il en existe une du fait du caractère aléatoire mais permettent de générer des chemins pouvant tenir compte de la cinématique du robot en appliquant une règle particulière lors de la génération des noeuds.

2.2 Suivi de lisière

Le suivi de lisière se compose de deux modules séquentiels : un module de détection des lisières issu de l'INRAE [?] et un module de suivi des lisières détectées.

Le module de détection prend en entrée la carte de gradients seuillée pour les obstacles infranchissables fournie par le MNE, représentant une carte d'occupation 2D (carte encodant la présence d'obstacles, de manière probabiliste ou binaire), et donne en sortie au maximum deux lisières, une à gauche et une à droite. Il génère ensuite une trajectoire associée à une des deux lisières, en fonction du choix de l'utilisateur (suivre la lisière de droite ou de gauche) et l'envoie au module de suivi pour le calcul des commandes et leur envoi à la brique de supervision.

La méthode de détection active de lisière implémentée repose sur l'adaptation de la détection de forme Snake contour et l'algorithme « Ball-Pivoting ». Cette approche permet de capturer uniquement les contours extérieurs des structures les plus proches du robot. La carte d'occupation peut avoir une certaine épaisseur pour des structures non "pleines", comme une haie clairsemée. Dans ce cas, extraire uniquement le contour extérieur permet d'éviter le traitement des points internes et de prévenir l'influence de points d'arrière-plan plus denses.

L'algorithme de détection commence par la détection du départ de la lisière à gauche et à droite du robot. Pour cela, on translate un cercle jusqu'à ce que celui-ci intersecte un point. Ce point correspond au premier point de la lisière (Figure 19) :

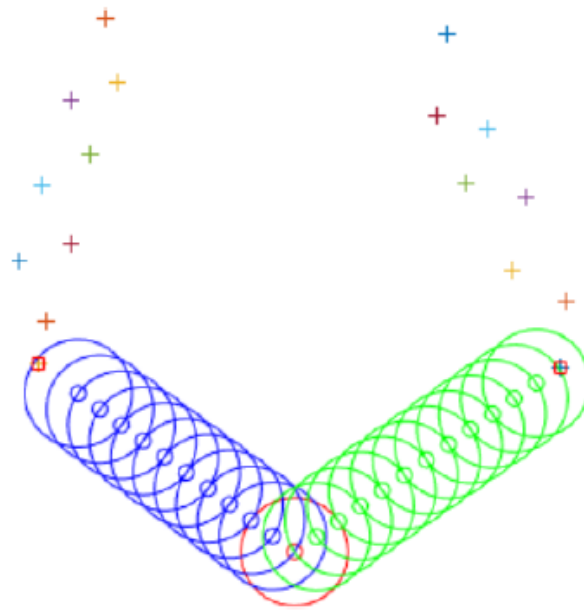


FIGURE 19 – Détection du premier point

Ensuite, on fait pivoter le cercle pour capturer l'ensemble des points de contour de la lisière (Figure 20) :

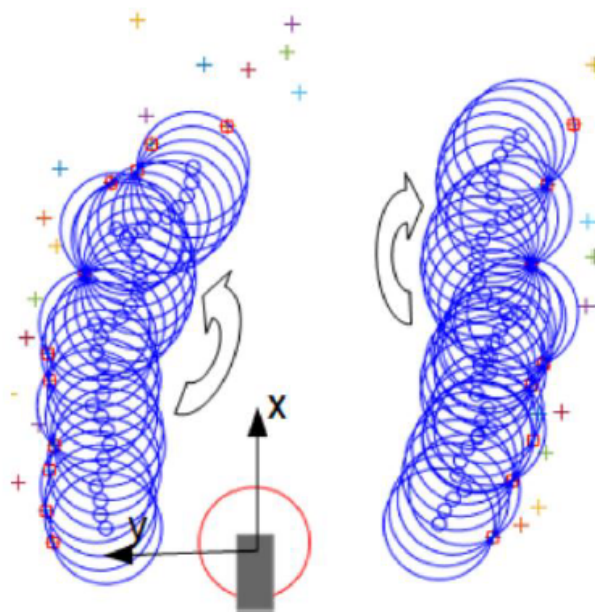


FIGURE 20 – Détection du contour de la lisière

Une fois les points de contour identifiés, des points intermédiaires sont ajoutés pour créer un nuage de points de chaque côté, formant une structure serpentine. Ces "serpents" représentent les bords estimés des lisières scannées (Figure 21) :

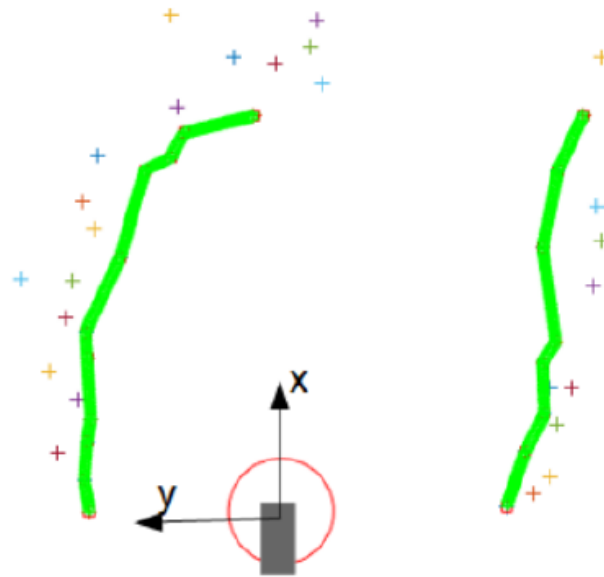


FIGURE 21 – Création du "serpent" à partir des points de contacts du cercle et des points intermédiaires

L'algorithme de détection de lisière est paramétrable. La taille du cercle a un grand impact sur la capacité de détection. Un cercle de petit diamètre permet d'obtenir un contour précis mais fonctionne mal avec les lisières naturelles qui possèdent de nombreuses aspérités. À l'inverse, un cercle de plus grande taille permet de passer au-dessus des aspérités et d'obtenir des lisières plus rectilignes. Les incréments de translation et de rotation du cercle sur le nuage de points permettent de maîtriser la précision du contour final. On peut également limiter la rotation du cercle sur le contour avec un angle maximal, afin d'éviter de détecter l'arrière de la lisière à la fin de cette dernière. Enfin, on limite la taille maximale du snake afin d'éviter des fausses détections et se concentrer sur la lisière en cours.

Une fois les lisières détectées, elles sont décalées suivant leur direction normale moyenne et envoyées à l'algorithme de suivi comme trajectoires à suivre.

Le module de suivi de trajectoire met en jeu un algorithme de suivi de chemin permettant une régulation latérale et longitudinale.

La régulation latérale est réalisée par un correcteur Pure-Pursuit. Ce correcteur a pour objectif de calculer un taux de lacet à partir d'un arc de cercle reliant le centre du repère du robot à un point mobile P placé sur le chemin, en avance de phase du projeté orthogonal H du point O sur le chemin. Pour augmenter la stabilité de la régulation et réduire les oscillations à haute vitesse, le point de visée anticipée P est placé à une distance curviligne l du point H , où l est proportionnel à la vitesse de suivi.

La régulation longitudinale est réalisée par un contrôleur qui détermine la vitesse de suivi en fonction de la vitesse consigne, de la distance à la fin du chemin et de la courbure du chemin en avance de phase. Cela permet de diminuer la vitesse à l'approche de virages ou de l'objectif.

Cette régulation est implémentée comme une étape de minimisation d'un critère (??) formulé comme une somme pondérée du carré de la différence de vitesse à la vitesse de référence

et de la moyenne de l'accélération centripète sur une portion du chemin à parcourir.

$$J(v) = \mu_1 \cdot (v_{max} - v)^2 + \mu_2 \frac{1}{n} \sum_{k=1}^n \frac{v^2}{R_{curv}(k)} \quad (3)$$

avec v la vitesse à optimiser en $m.s^{-1}$, v_{max} la vitesse de consigne en $m.s^{-1}$, n le nombre de points de la portion de chemin et $R_{curv}(k)$ le rayon de courbure en m du chemin au point k .

3 Briques de contrôle

L'objectif des briques de contrôle est de retourner des commandes, ici de vitesses linéaires et angulaires, pour suivre un chemin donné en entrée en tenant compte de certains critères.

3.1 Retour sur traces et rejeu de trajectoire

Le retour sur traces et le rejeu de trajectoire sont assurés par la même brique, permettant d'enregistrer en temps réel la trajectoire du robot pour ensuite l'envoyer en entrée du même module de contrôle utilisé par le suivi de lisière avec un paramétrage spécifique pour fournir les commandes de déplacement à la brique de supervision.

Pour assurer le déplacement dans un sens ou dans l'autre, il suffit d'inverser ou non les points du chemin, les consignes de vitesse et le cap du robot.

3.2 Planification locale

La planification locale se fait dans l'espace de la commande projetée en vue de dessus ($v_{2d}, \dot{\alpha}_z$), v_{2d} étant la vitesse linéaire et $\dot{\alpha}_z$ la vitesse angulaire selon l'axe Z en 2D, en se servant de la carte de gradients seuillée pour les obstacles non franchissables.

L'algorithme de planification utilisé est une adaptation de la Dynamic Window Approach (DWA) [?]. La méthode repose sur l'exploration de l'espace de commande afin de découvrir des combinaisons appropriées par la formulation d'un problème d'optimisation sur un horizon temporel glissant, plus étendu que l'intervalle de commande. L'avantage d'une telle approche pour la planification locale réside dans l'admissibilité cinématique des commandes tandis qu'une approche se limitant à l'espace de la tâche peut engendrer des chemins non praticables. Par exemple, l'utilisation de la planification donnée par le A* basée sur un environnement discrétisé et des champs de potentiels peut produire des inflexions abruptes dans un chemin qu'un robot non holonome ne peut suivre : le robot est incapable d'effectuer "un pas de côté" pour éviter un obstacle.

D'autres alternatives de la DWA existent [?, ?] mais les recherches récentes tentent d'intégrer les contraintes de cinématique et la notion de risque directement dans la carte [?, ?, ?, ?] ou d'utiliser l'apprentissage pour calculer les commandes optimales à exécuter [?]. Cependant, ces algorithmes sont plus coûteux en temps de calcul.

Des primitives de déplacement de type "arc" en vue de dessus sont évaluées sur un horizon temporel de 2 s. Chaque arc est paramétré par une paire de vitesses ($v_{2d}, \dot{\alpha}_z$) constantes sur la fenêtre temporelle.

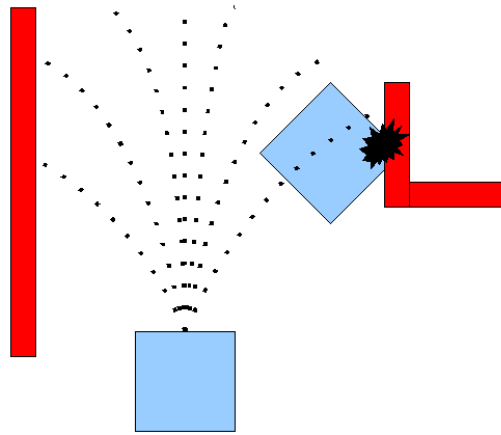


FIGURE 22 – Illustration des trajectoires évaluées par la planification locale DWA

Le principe de base de l'algorithme « Dynamic Window Approach » (DWA) est le suivant :

- Effectuer un échantillonnage discret dans l'espace de contrôle du robot (dx , dy , $d\theta$).
- Pour chaque vitesse échantillonnée, effectuer une simulation vers l'avant (Figure 22) à partir de l'état actuel du robot afin de prédire ce qui se passerait si cette vitesse était appliquée pendant une (courte) période.
- Évaluer (noter) chaque trajectoire résultant de la simulation vers l'avant, à l'aide d'un indicateur intégrant des caractéristiques telles que : la proximité des obstacles, la proximité de l'objectif, la proximité de la trajectoire globale et la vitesse. Écarter les trajectoires illégales (celles qui entrent en collision avec des obstacles).
- Choisir la trajectoire ayant obtenu le meilleur score et envoyer la vitesse associée à la base mobile.
- Répéter l'opération.

Le critère optimisé est un critère élaboré par SHERPA Engineering, fonction de l'écart par rapport à la vitesse consigne, de la direction / orientation du robot par rapport à l'objectif et de la présence d'obstacles. Il apparaît que selon la configuration d'obstacles, il n'y a aucune raison pour que ce critère soit convexe. Il est donc optimisé avec une approche en force brute de type grid search. Cette approche consiste à discrétiser l'espace de commande et calculer le coût associé à chaque paire de commandes. La paire avec le plus faible coût étant la commande optimale pour la discrétisation donnée. Une fois la paire optimale trouvée, on calcule les commandes à envoyer au robot (v, ω_z) pour les adapter à la topologie du terrain dans le repère 3D.

L'évaluation du terme de la présence des obstacles est lourd en calculs car il demande de générer l'empreinte du robot sur $2s$ pour chaque paire de commandes. L'empreinte pour une paire de commande étant invariante, ces dernières sont générées en amont du lancement de la Brique MOBILEX* et stockées sous forme de Look-Up Tables. Lors du lancement de la brique de planification locale, elles sont chargées en mémoire vive, permettant de gagner un temps considérable pour trouver la commande optimale à appliquer. La superposition de l'empreinte et de la carte d'obstacles dans le repère du robot permet d'évaluer la présence d'obstacles sur la trajectoire générée.

3.3 Asservissement du taux de lacet

Le Barakuda est contrôlé en fournissant une consigne de vitesse identique aux roues d'un même côté, lui conférant ainsi une cinématique de type "skid steer". Avec ce type de cinématique, les roues avant et arrières dérapent nécessairement pour que le robot puisse tourner, contrairement à une cinématique à deux roues différentielles et une roue libre majoritairement employée en robotique indoor. Par conséquent, on ne peut pas faire l'hypothèse que le robot réalise vraiment le taux de rotation souhaité avec les vitesses de roue envoyées à gauche et droite en utilisant les équations d'un modèle cinématique différentiel à 2 roues sans glissement.

Une solution simple et couramment employée pour résoudre ce problème consiste à asservir le taux de lacet pour que ce dernier corresponde bien aux commandes de haut niveau. Pour ce faire, on utilise les données odométriques fournies par le capteur INS pour obtenir le taux de lacet à tout instant.

La correction implémentée est un simple correcteur Proportionnel Intégral avec un gain FeedForward. Le gain FeedForward joue le rôle d'un précompensateur trivial : inversion du gain $K = \lim_{s \rightarrow 0} H(s)$ où H est la fonction de transfert du système linéarisé. Le gain proportionnel fait l'essentiel de la régulation tandis que le gain intégral limite l'écart statique et l'effet des perturbations. Le gain feedforward augmente les performances en asservissement (précompensateur) en ramenant le système à un système à gain unitaire, tandis que le correcteur Proportionnel Intégral fait la régulation.

Avec un modèle linéarisé $H(s)$ du Barakuda, il serait théoriquement possible d'augmenter les performances en régulation et ainsi réduire l'erreur de traînage (pour une consigne dynamique) sur le taux de lacet en remplaçant ce correcteur par un correcteur de type RST [?] que nous pourrions paramétrer pour obtenir un comportement souhaité de manière plus précise.

4 Autres briques

4.1 Interface avec le Barakuda

Cette brique est responsable de la communication avec le Barakuda via les trames UDP. D'une part, les informations remontées par le robot sont distribuées au reste du système sous forme de diagnostics. D'autre part, les consignes corrigées par le correcteur cinématique sont envoyées au robot, selon les formules habituelles (??) :

$$\Omega_{roues} = S \left(v_x \pm \omega_z \frac{L}{2} \right) \quad (4)$$

avec Ω_{roues} la vitesse de rotation des variateurs en rpm , v_x la vitesse linéaire souhaitée en $m.s^{-1}$, ω_z la vitesse angulaire souhaitée en $rad.s^{-1}$, L la voie (largeur) du véhicule en m (ici, 0.95 m), et S un facteur d'échelle dépendant des variateurs (ici, 1000 $rpm.s.m^{-1}$).

4.2 Téléopération à la manette

Cette simple brique lit et convertit les positions du joystick en commandes linéaires et angulaires avec un facteur d'échelle pour les envoyer directement au robot. L'ajout de la planification locale avec une configuration particulière entre cette brique et le robot permet de réaliser le

mode safety first, empêchant le robot d'entrer dans des obstacles en modifiant légèrement les commandes problématiques transmises ou en arrêtant le robot.

4.3 Brique de supervision

Cette brique récupère l'ensemble des commandes fournies par les fonctions de pilotage et permet à l'opérateur de choisir laquelle est envoyée au robot via une liste déroulante dans l'IHM.

Cette brique est également responsable d'une partie de la sécurité du robot puisqu'elle peut limiter la vitesse de ce dernier tout en gardant la même trajectoire lorsque la commande reçue provoque un dépassement du seuil de l'accélération centrifuge autorisé ou encore provoquer l'arrêt et le serrage des freins lors d'un dysfonctionnement grave retourné par une brique. La commande vérifiée ou limitée est ensuite envoyée au correcteur de taux de lacet.

4.4 Retour visuel annoté pour l'opérateur

Cette brique utilise le retour image de la caméra frontale du système de perception pour y superposer différentes informations, éventuellement agrégées à partir des autres modules.

Afin de respecter les exigences du challenge Mobilex*, les informations suivantes sont superposées au flux image :

- Affichage type “Viseur Tête Haute” avec les angles de roulis, tangage, et lacet.
- Tracé des chemins de navigation trouvés par la planification globale en 3D.
- Tracé du chemin local à court terme (2 s), étant donné la commande actuelle.
- Représentation des obstacles avec des régions colorées en fonction de la distance.
- Représentation colorée des flaques d'eau et végétation traversable
- Représentation des waypoints en 3D dans l'image.
- Compas

La projection dans l'image des informations définies dans le repère monde a été réalisée à l'aide de la calibration intrinsèque de la caméra ainsi que de la calibration extrinsèque caméra/LIDAR* réalisée auparavant et du système de transformations tf de ROS* permettant de définir une matrice de transformation homogène 4x4 pour projeter les éléments du repère monde vers le repère image. Cette transformation est utilisée pour la représentation des obstacles, des waypoints, des flaques d'eau, des végétations traversables et des chemins.

On obtient alors le retour annoté Figure 23 :

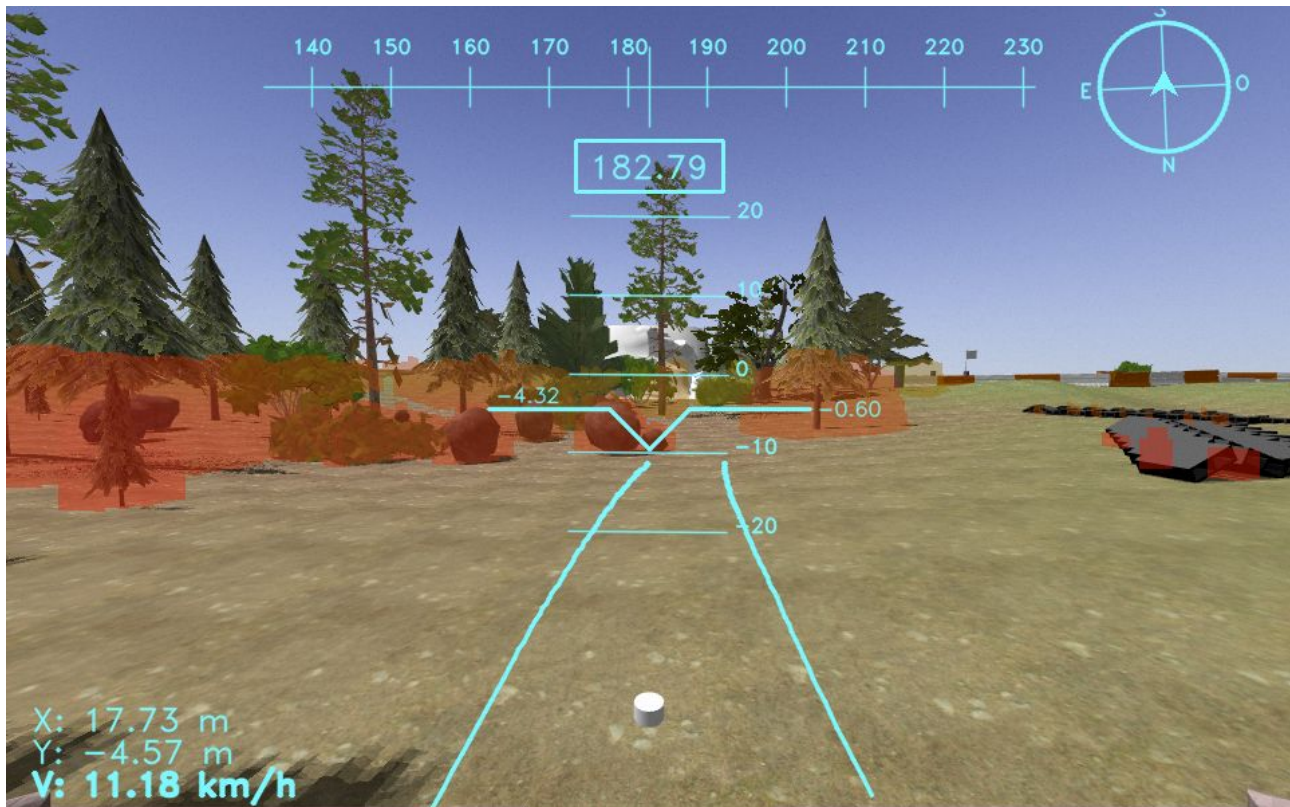


FIGURE 23 – Retour visuel annoté en simulation

4.5 IHM

Enfin, la brique de l'IHM fournit une interface graphique (Figure 24) sur le PC débarqué à l'aide du framework ROS* RQT, permettant au téléopérateur d'accéder au retour visuel annoté, de donner des ordres de mission au robot tout en pouvant modifier les paramètres à sa guise, de sauvegarder des trajectoires pour les réutiliser lors des modes rejou et retour sur traces, de pouvoir entrer des coordonnées de points à atteindre et accéder à un ensemble de données et de diagnostics pour du debug.

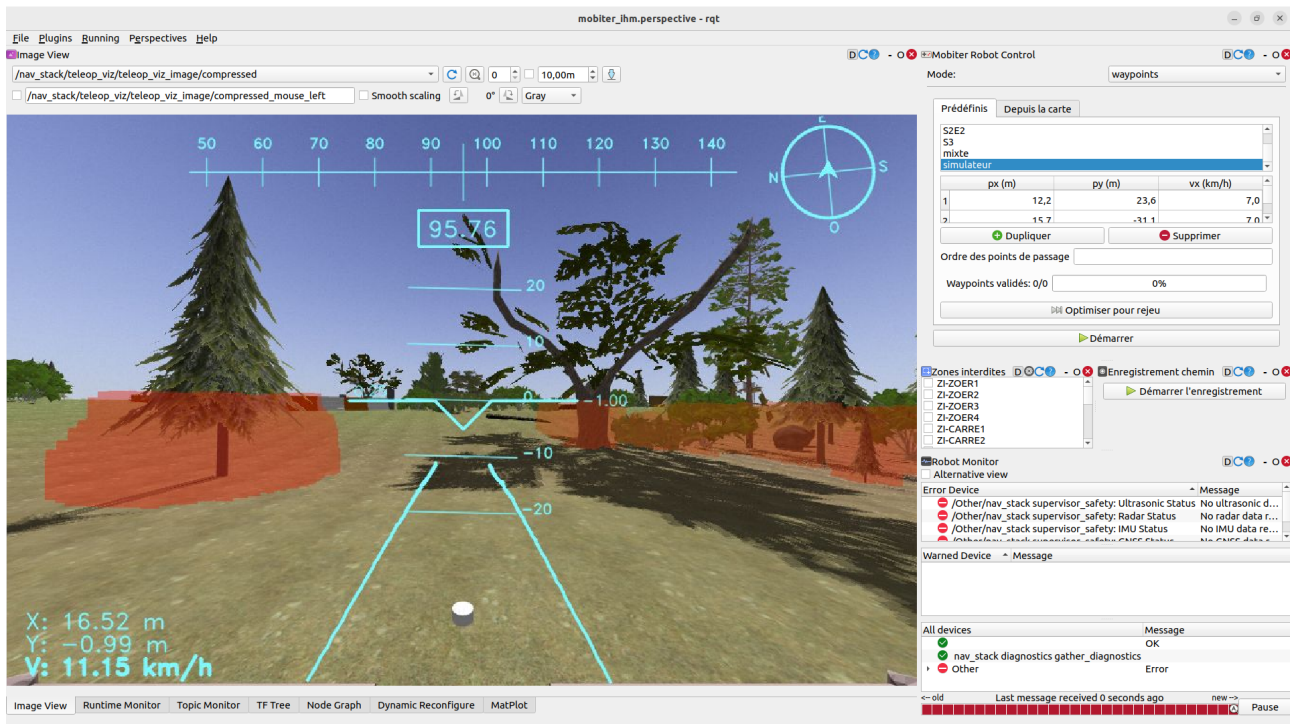


FIGURE 24 – Interface RQT

En parallèle de l'interface RQT, une visualisation RVIZ (Figure 25) peut être lancée, permettant de visualiser les différentes cartes générées par la brique du MNE, les chemins planifiés et permettant de renseigner un point à atteindre directement sur la carte.

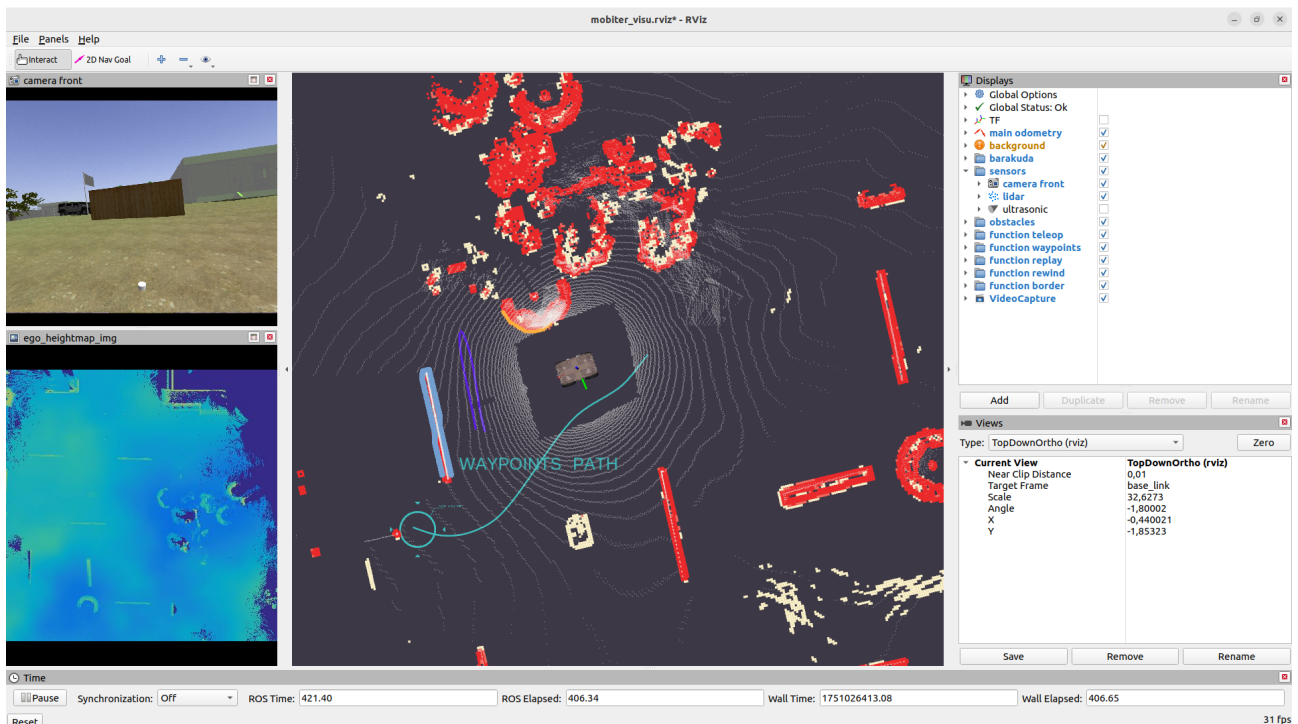


FIGURE 25 – Visualisation RVIZ

IV Le stage

1 Les missions

J'ai été recruté comme stagiaire chez SHERPA Engineering afin d'apporter une aide à l'intégration et aux tests terrain pour compte du défi 3 du Challenge MOBILEX*.

Ma mission principale consiste à participer aux tests des différentes briques logicielles développées sur le robot. Les tests se font d'abord en simulation, puis, lorsque les résultats sont concluants, sur le terrain. Lors des tests terrains, il est nécessaire de prévoir une série de tests pertinents pour mettre à l'épreuve les briques sous test et de communiquer avec l'équipe de l'INRAE pour programmer une date et prévoir la disponibilité du matériel à utiliser pour les tests. Une fois les tests réalisés, un compte-rendu est fait à l'équipe de développement.

En cas de problèmes lors des tests, il est impératif de noter les circonstances dans lesquelles ils sont apparus, trouver des scénarios reproduisant ces derniers, réaliser des diagnostics, isoler la cause et, dans la mesure du possible, proposer une solution de résolution.

En dehors des sessions de tests planifiées, mes missions se sont articulées autour de deux axes principaux :

- **L'automatisation du post-traitement** : J'ai développé un générateur automatique de rapports d'essais. Cet outil synthétise les données d'exécution afin de fournir une analyse visuelle et rapide du déroulement des tests.
- **La standardisation des procédures** : J'ai mis sur pied un protocole et un plan de test rigoureux. Ce cadre méthodologique vise d'une part à évaluer la conformité du système vis-à-vis des exigences fonctionnelles spécifiées par la DGA pour le Défi 3, et d'autre part à garantir la répétabilité des scénarios d'essai, indépendamment de l'ingénieur chargé de l'exécution.

Enfin, mon avis est attendu lors des questionnements stratégiques.

2 Présentation des travaux réalisés

2.1 Prise en main du projet

Lors de mon arrivée à SHERPA Engineering, il a été indispensable pour moi de prendre en main un projet débuté depuis plus de deux ans.

Mon premier objectif a consisté à cloner le dépôt Git de l'ensemble du projet Mobiter sur notre poste de travail. Ce dernier fonctionne sous Ubuntu 22.04 et intègre Distrobox, un outil de conteneurisation permettant d'exécuter une surcouche logicielle sous un autre système d'exploitation — en l'occurrence Ubuntu 20.04. Cette configuration est indispensable pour utiliser ROS Noetic. En effet, l'intégralité de la brique logicielle MOBILEX a été développée sous cette version de ROS, qui s'avère incompatible avec la distribution native Ubuntu 22.04.

Une fois le simulateur fonctionnel, j'ai alors pu me familiariser avec l'utilisation de l'IHM et les différents modes de conduite disponibles. Puis afin de pouvoir proposer des tests pertinents

et comprendre l'architecture logicielle de la brique MOBILEX*, aussi bien d'un point de vue théorique que technique, il m'a fallu consulter l'ensemble des codes.

2.2 Intégration et évaluation du slam DLIO

Au début de mon stage, l'équipe Mobiter avait engagé des travaux de refonte du module de SLAM (Simultaneous Localization and Mapping). Lors des défis précédents, la solution logicielle reposait exclusivement sur le Kitware SLAM, une approche purement LiDAR. Celle-ci présentait de graves limites dans les environnements très dégagés (comme les grands espaces extérieurs), où la rareté des caractéristiques géométriques et des retours LiDAR nuisait à la qualité de la localisation. De plus, ce composant représentait l'une des charges computationnelles les plus lourdes de la pile logicielle du projet. Afin de pallier ces défauts, le SLAM DLIO (Direct LiDAR-Inertial Odometry), combinant les données LiDAR et une centrale inertielle (IMU), a été choisi comme une alternative plus robuste pour la navigation en milieux ouverts.

C'est dans ce contexte que j'ai participé à l'intégration du SLAM DLIO au sein de la brique MOBILEX*, puis de contribuer à son évaluation sur le terrain. Une fois cette intégration logicielle finalisée, nous avons mené une campagne de tests afin de valider son bon fonctionnement et de quantifier ses performances (en matière de dérive et de charge CPU) par rapport au Kitware SLAM et à la vérité terrain.(Figure 26)

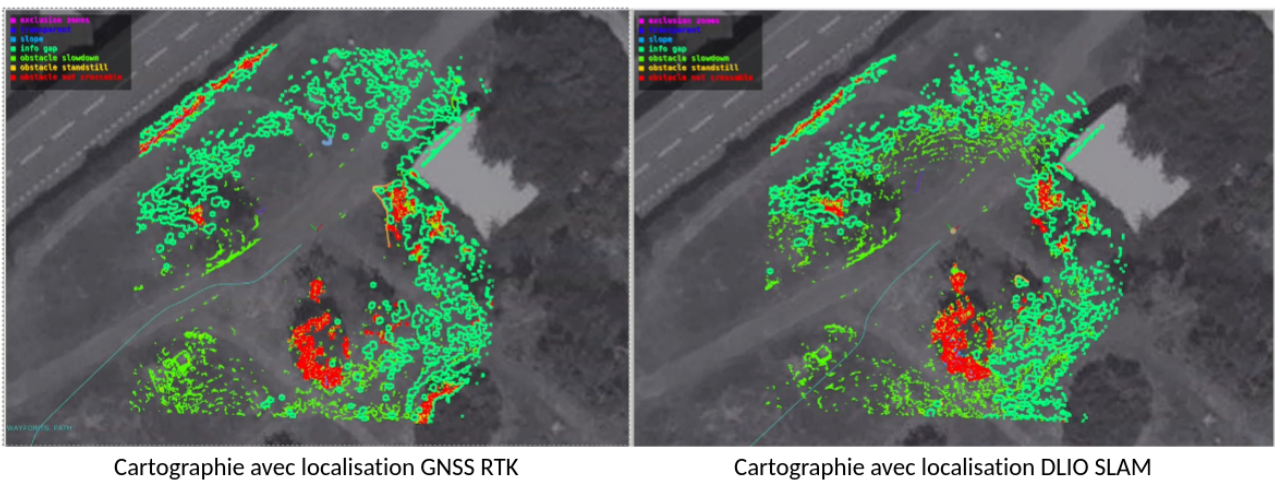


FIGURE 26 – Comparaison de cartographie en GNSS RTK VS en DLIO

Afin d'évaluer les performances du SLAM DLIO, des essais ont été menés sur le site de l'INRAE en comparant les trajectoires générées par ce dernier et par le Kitware SLAM avec la vérité terrain fournie par un système GNSS RTK. La méthodologie a consisté, dans un premier temps, à enregistrer un fichier de données (ROS bag) lors d'une séquence de suivi de waypoints avec le SLAM DLIO actif, tout en enregistrant la position de référence renvoyée par la centrale de navigation SBG Systems. Dans un second temps, ce bag a été rejoué hors ligne afin d'exécuter le Kitware SLAM et de générer un enregistrement unique centralisant les trois modes de localisation (GNSS RTK, DLIO et Kitware). Enfin, l'utilisation des outils de la bibliothèque EVO nous a permis de comparer et d'évaluer précisément les performances des deux algorithmes de SLAM par rapport à la vérité terrain.(Figure 27 et Figure 28)

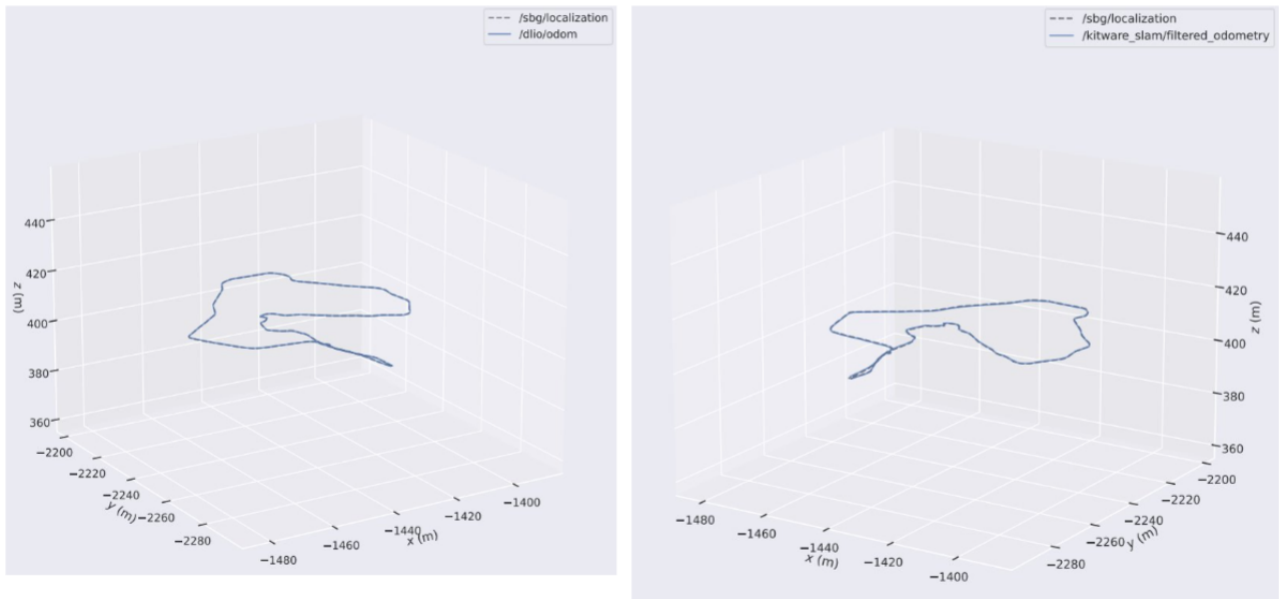


FIGURE 27 – Comparaison des trajectoires en DLIO / Kitware par rapport à la vérité terrain

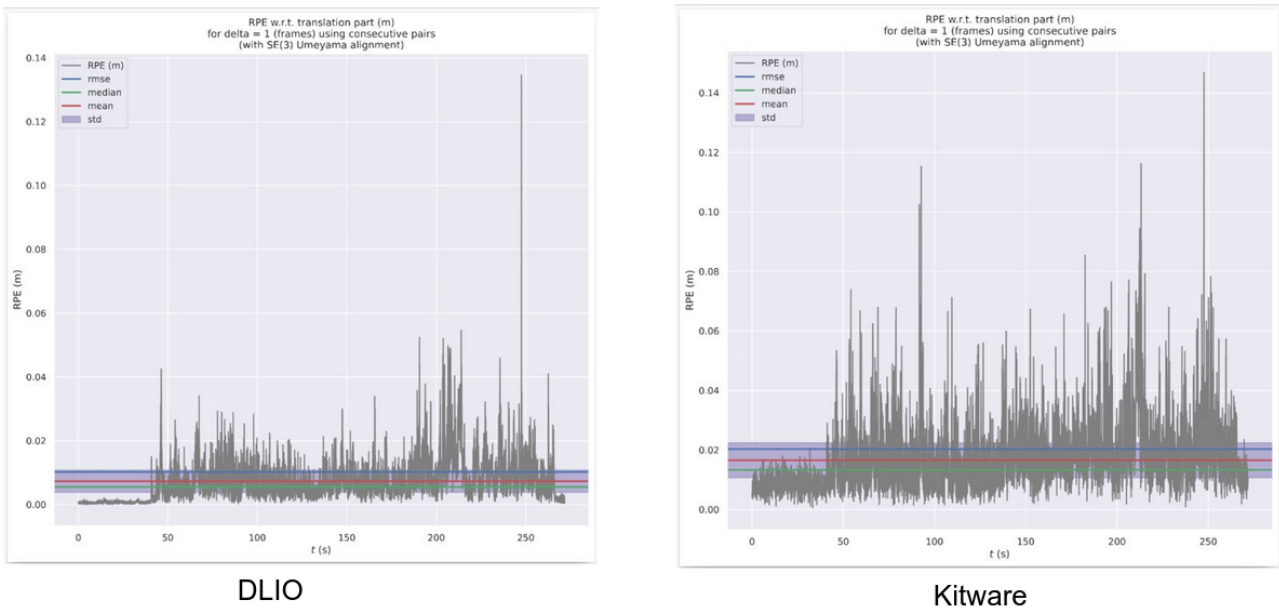


FIGURE 28 – Comparaison des erreurs relatives de position en DLIO vs en Kitware

2.3 Évaluation de la dérive du SLAM DLIO par rapport à la vérité terrain

La bibliothèque python EVO utilise l'algorithme de Kabsch-Umeyama [?] pour aligner les trajectoires avant comparaison et évaluation. Cet algorithme permet de trouver la meilleure transformation rigide (rotation et translation) qui minimise l'écart quadratique moyen (RMSD) entre deux ensembles de points appariés. Il est utile pour l'alignement d'ensembles de points en infographie, ainsi qu'en chimio-informatique et en bio-informatique pour comparer des structures moléculaires et protéiques.

Cette étape de transformation est cruciale pour s'assurer que les trajectoires sont comparées dans le même référentiel, éliminant ainsi les effets de décalage initial ou de rotation. Une fois les nuage de points alignés, EVO compare les trajectoires et retourne les métriques

suivantes :

- **max** : c'est la valeur de l'erreur la plus élevée enregistrée sur l'ensemble de la trajectoire.
- **mean** : c'est la moyenne arithmétique simple de toutes les erreurs de pose calculées.
- **median** : c'est la valeur centrale. Exactement la moitié des poses ont une erreur inférieure à la médiane, et l'autre moitié a une erreur supérieure.
- **min** : c'est la valeur de l'erreur la plus faible enregistrée sur l'ensemble de la trajectoire.
- **rmse** (Root Mean Square Error - Erreur Quadratique Moyenne) : c'est la racine carrée de la moyenne des carrés des erreurs.
- **sse** (Sum of Squared Errors - Somme des Erreurs Quadratiques) : c'est la somme de toutes les erreurs individuelles élevées au carré.
- **std** (Standard Deviation - Écart Type) : c'est une mesure de la dispersion des erreurs autour de la moyenne (mean).

Afin de qualifier les performances du SLAM DLIO, il était indispensable d'évaluer sa dérive par rapport à la vérité terrain en fonction de la distance parcourue. Pour cela, à partir d'un bag de données ROS contenant les trajectoires (SLAM et vérité terrain) enregistrées lors d'essais réels, nous avons implémenté un script qui calcule l'erreur euclidienne de position, la distance réelle parcourue puis la dérive du SLAM à chaque instant. La dérive est donnée en pourcentage par la formule suivante (??) :

$$\text{Dérive}[k] = \frac{100 \cdot \sqrt{(x_{\text{slam}}[k] - x_{\text{sbg}}[k])^2 + (y_{\text{slam}}[k] - y_{\text{sbg}}[k])^2 + (z_{\text{slam}}[k] - z_{\text{sbg}}[k])^2}}{d[k]} \quad (5)$$

avec $x_{\text{slam}}[k]$, $y_{\text{slam}}[k]$, $z_{\text{slam}}[k]$ les coordonnées de la position estimée par le SLAM à l'instant k , $x_{\text{sbg}}[k]$, $y_{\text{sbg}}[k]$, $z_{\text{sbg}}[k]$ les coordonnées de la vérité terrain fournie par la centrale SBG, et $d[k]$ la distance cumulée parcourue jusqu'à l'instant k , calculée de manière incrémentale selon la formule (??) : avec $d[0] = 0$.

$$d[k] = d[k-1] + \sqrt{(x_{\text{sbg}}[k] - x_{\text{sbg}}[k-1])^2 + (y_{\text{sbg}}[k] - y_{\text{sbg}}[k-1])^2 + (z_{\text{sbg}}[k] - z_{\text{sbg}}[k-1])^2} \quad (6)$$

Une étape primordiale avant d'appliquer le calcul de la dérive consistait à s'assurer que les trajectoires SLAM et SBG (vérité terrain) soient dans le même repère et que, pour chaque instant k (timestamp), les positions estimées par le SLAM et la vérité terrain soient appariées. Or, les deux trajectoires enregistrées dans le bag n'étaient pas parfaitement synchronisées ; nous avons donc dû les synchroniser en utilisant une fonction d'interpolation linéaire entre deux points successifs (??). Cette fonction permet d'interpoler les données d'une trajectoire pour obtenir des valeurs correspondantes aux timestamps de l'autre trajectoire. Ainsi, nous avons pu aligner les deux ensembles de données et garantir que chaque position estimée par le SLAM correspondait à une position de vérité terrain à un instant donné.

$$f(t) = f(t_k) + \frac{t - t_k}{t_{k+1} - t_k} \cdot (f(t_{k+1}) - f(t_k)), \quad t_k \leq t \leq t_{k+1} \quad (7)$$

avec $f(t)$ la valeur interpolée à l'instant t , t_k et t_{k+1} les deux timestamps successifs encadrant t , et $f(t_k)$, $f(t_{k+1})$ les valeurs connues aux instants t_k et t_{k+1} . Cette formule est appliquée

indépendamment sur chaque coordonnée (x, y, z) .

La figure Figure 29 illustre l'évolution de la dérive du SLAM DLIO par rapport à la vérité terrain en fonction de la distance parcourue. On observe que la dérive reste relativement faible, avec une valeur maximale d'environ 1,5% sur l'ensemble de la trajectoire. Cette performance est significativement meilleure que celle du Kitware SLAM, qui présente une dérive plus importante, surtout dans les zones dégagées où les caractéristiques géométriques sont rares. Ces résultats confirment que le SLAM DLIO est une solution plus robuste et fiable pour la navigation en milieux ouverts, offrant une meilleure précision de localisation et une charge computationnelle réduite.

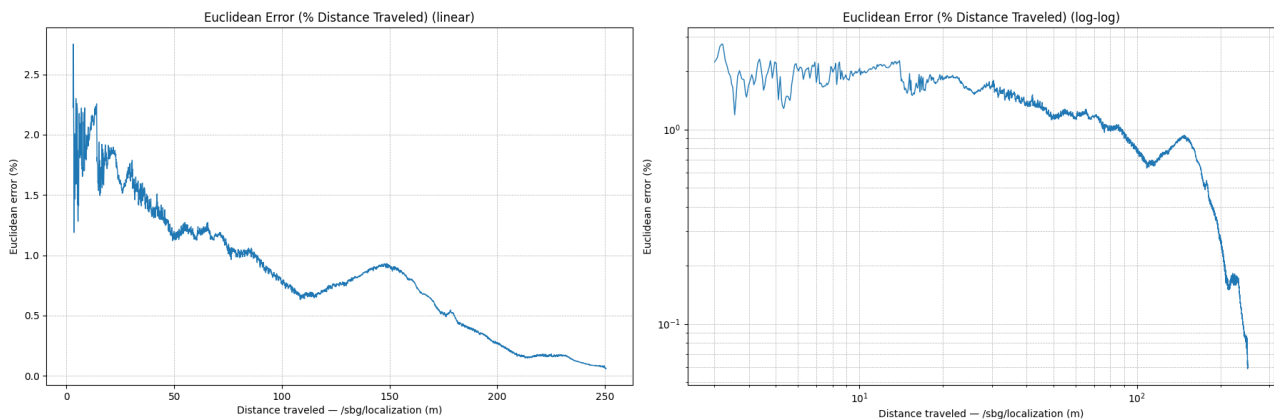


FIGURE 29 – Évolution de la dérive du SLAM DLIO par rapport à la vérité terrain en fonction de la distance parcourue

2.4 Mise en oeuvre du générateur automatique de rapports d'essais

L'une de mes missions principales a été de développer un générateur automatique de rapports d'essais. Cet outil vise à synthétiser les données d'exécution des tests pour fournir une analyse visuelle et rapide du déroulement des essais afin de faciliter les diagnostics.

Le générateur de rapports d'essais a été conçu pour assurer un traitement offline des fichiers de données (ROS bag) enregistrés lors des tests. Il extrait les informations pertinentes, telles que les trajectoires, les vitesses, les erreurs de localisation, les événements critiques, les images de cartes construites et de visualisation, puis génère un rapport structuré comprenant des graphiques, des informations de diagnostic et des vidéos.

Après discussion avec l'ensemble de l'équipe, une liste d'éléments pertinents à inclure dans le rapport de test a été établie. Cette liste comprend notamment :

- La liste des diagnostics (avec timestamps) retournés sur l'IHM durant toute la durée des tests : sous forme de figure facilement interpretable puis sous forme de test plus descriptif contenant le message d'erreur associé à chaque diagnostic.
- Le mode de control du robot (téléopération, suivie de waypoint, ...) ainsi que le mode de localisation (GNSS, SLAM) du robot à chaque timestamps sous forme de figure.

- Un tracé des positions linéaires et angulaires du robot.
- Un tracé de la trajectoire du robot en slam et en GNSS RTK avec les waypoints à raliés, ainsi qu'une évaluation des erreurs de localisation.
- Un tracé de la trajectoire du robot avec les waypoints à raliés ainsi qu'une représentation du mode de localisation sur chaque section de la trajectoire.
- Un tracé des erreurs de vitesses linéaires sur toute la trajectoire par rapport aux vitesses de consigne souhaitée.
- Des vidéos synchronisées par rapport aux timestamps des bag, montrant la construction du MNE (Modèle Numérique d'Elevation), de la planification globale, de la planification locale, de la carte des gradients ainsi que du retour vidéo sur l'IHM.

L'outil a été développé en Python et utilise des bibliothèques telles que Matplotlib pour la visualisation des données, Pandas pour le traitement des données et OpenCV pour la manipulation d'images et de vidéos. A partir du bag ROS, le rapport final est généré au format html sous demande des membres de l'équipe. L'organigramme de l'outil est présenté dans la (Figure 30) :

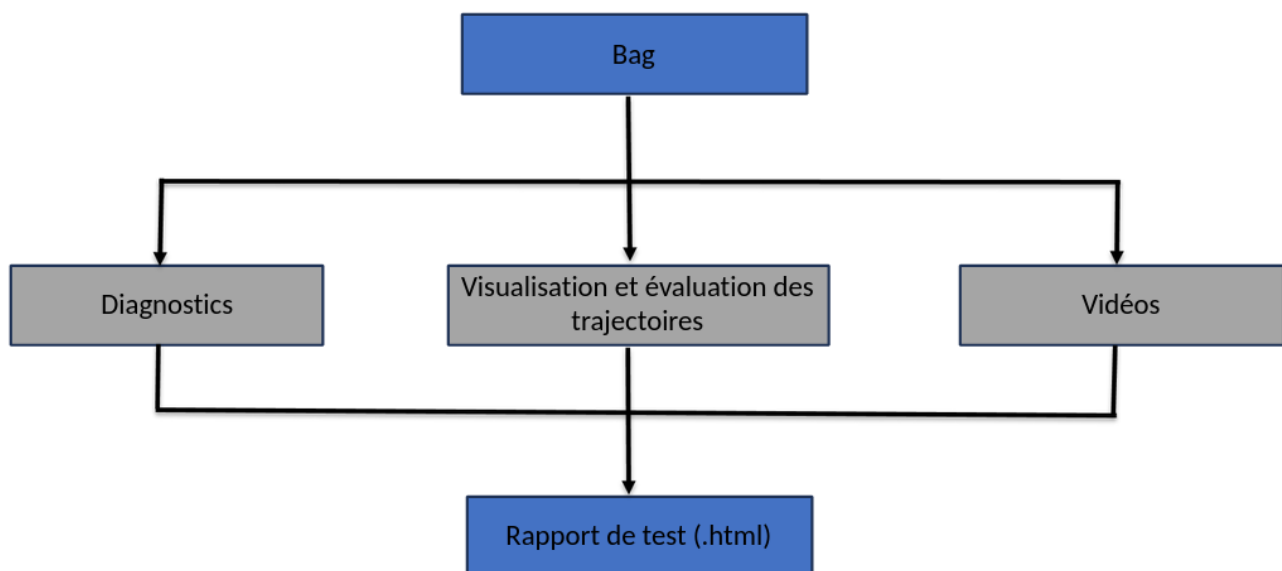


FIGURE 30 – Organigramme de l'outil de génération automatique de rapport de test

2.4.1 Extraction et génération des diagnostics de l'IHM

L'objectif principal de ce module est d'obtenir une vision consolidée et exhaustive de l'ensemble des diagnostics émis par l'interface homme-machine (IHM) sur la totalité de la plage temporelle de l'essai. Cette approche permet notamment de capturer et de consigner les événements transitoires (des alertes fugaces qui apparaissent et disparaissent rapidement), évitant ainsi qu'ils ne soient omis par l'opérateur lors du déroulement du test en temps réel. Pour ce faire, l'outil génère plusieurs représentations graphiques, dont une frise chronologique (timeline) des diagnostics (Figure 31), indexée par identifiant d'alerte. Cette visualisation temporelle permet d'identifier immédiatement les éléments où des anomalies sont survenues, puis de fournir un résumé sur ces diagnostics anormaux détectés. Au sein de la pile logicielle (stack) MOBITER,

les diagnostics sont hiérarchisés et catégorisés en quatre grands groupes fonctionnels (niveau) :

- **OK** : qui indique que tout fonctionne correctement.
- **WARNING** : qui indique un avertissement ou un problème potentiel.
- **ERROR** : qui indique une erreur nécessitant une attention immédiate.
- **STALE** : qui indique que les informations ne sont plus à jour.

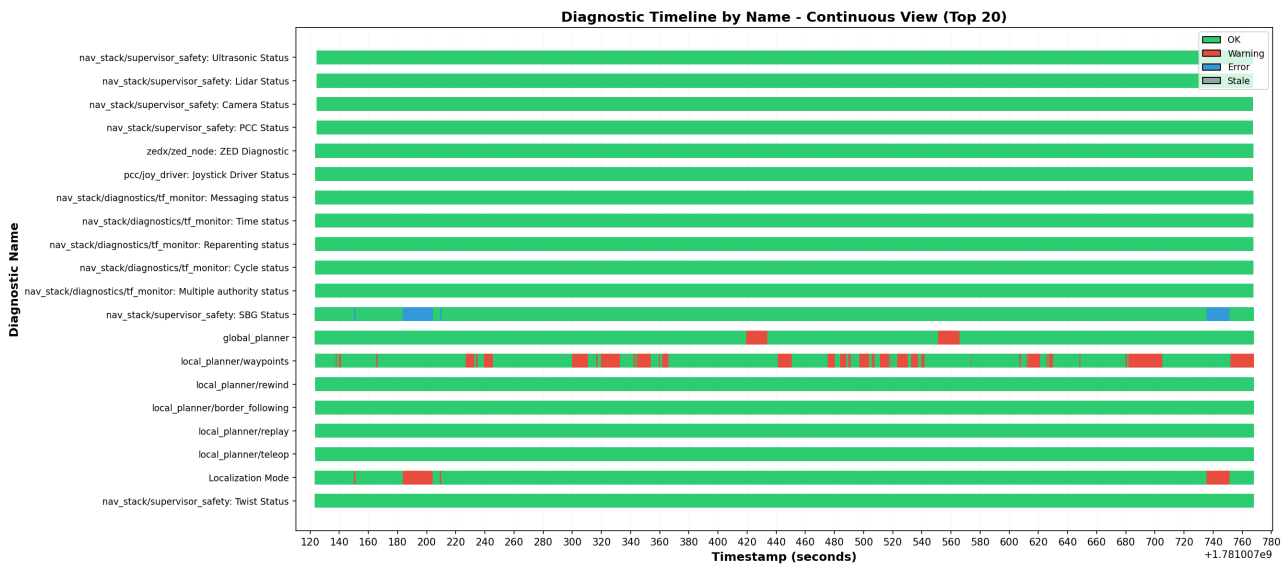


FIGURE 31 – Timeline des diagnostics de l'IHM par nom de diagnostic

Deplus, pour des soucis de debug il est souvent interessant de savoir à quel moment le robot a changé de mode de contrôle (téléopération, suivis de waypoint, suivis de lisiere) ou de localisation (GNSS ou SLAM). L'outil génère donc également des timelines des changements de mode de contrôle et de localisation (Figure 32 et Figure 33).

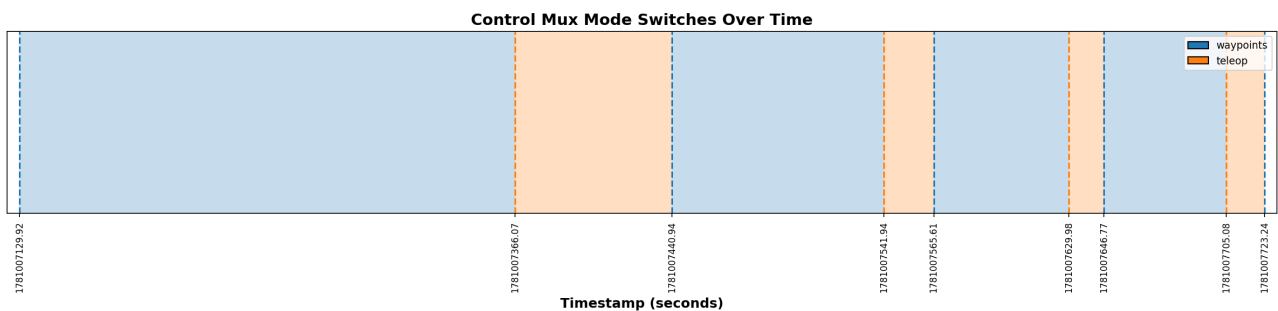


FIGURE 32 – Timeline des changements de mode de contrôle

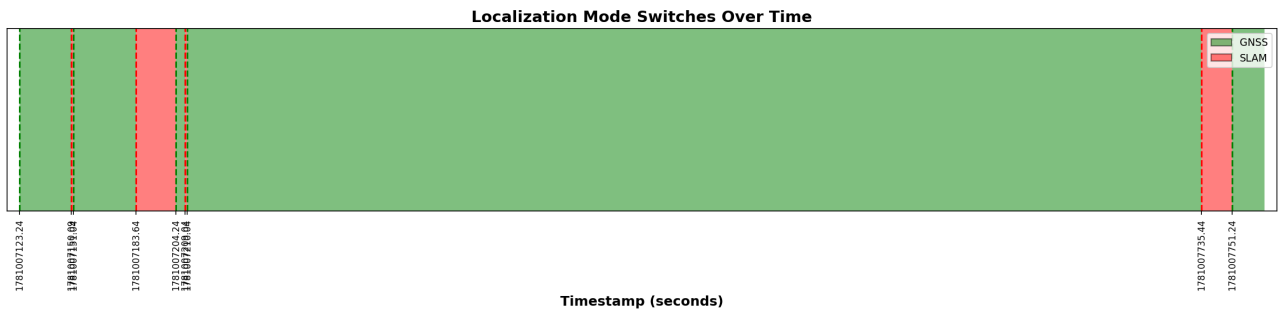


FIGURE 33 – Timeline des changements de mode de localisation

Le processus débute par l'analyse du fichier d'enregistrement (bag file), dont l'objectif est d'extraire l'ensemble des topics de type **DiagnosticArray**. Si une liste restrictive de topics est fournie en paramètre, l'extraction se limite à ces derniers ; dans le cas contraire, l'intégralité des flux de type **DiagnosticArray** est récupérée par défaut. Une fois l'extraction effectuée, les informations utiles sont structurées et stockées chronologiquement au sein d'un dictionnaire indexé par horodatage (timestamp). À partir de cette structure de données, des scripts exploitant des library Python (pandas, numpy, matplotlib) génèrent automatiquement l'ensemble des figures d'analyse présentées précédemment, ainsi qu'un rapport de synthèse textuel (format .txt). Enfin, afin de mettre en relation les anomalies matérielles ou logicielles avec l'état opérationnel du robot, l'outil extrait également les messages du topic **nav_stack/control_mux/mode**. L'analyse de ces données permet de tracer et de corréliser précisément les changements de modes de contrôle du système au cours de l'essai. La figure (Figure 34) suivante illustre l'organigramme de cet outil :

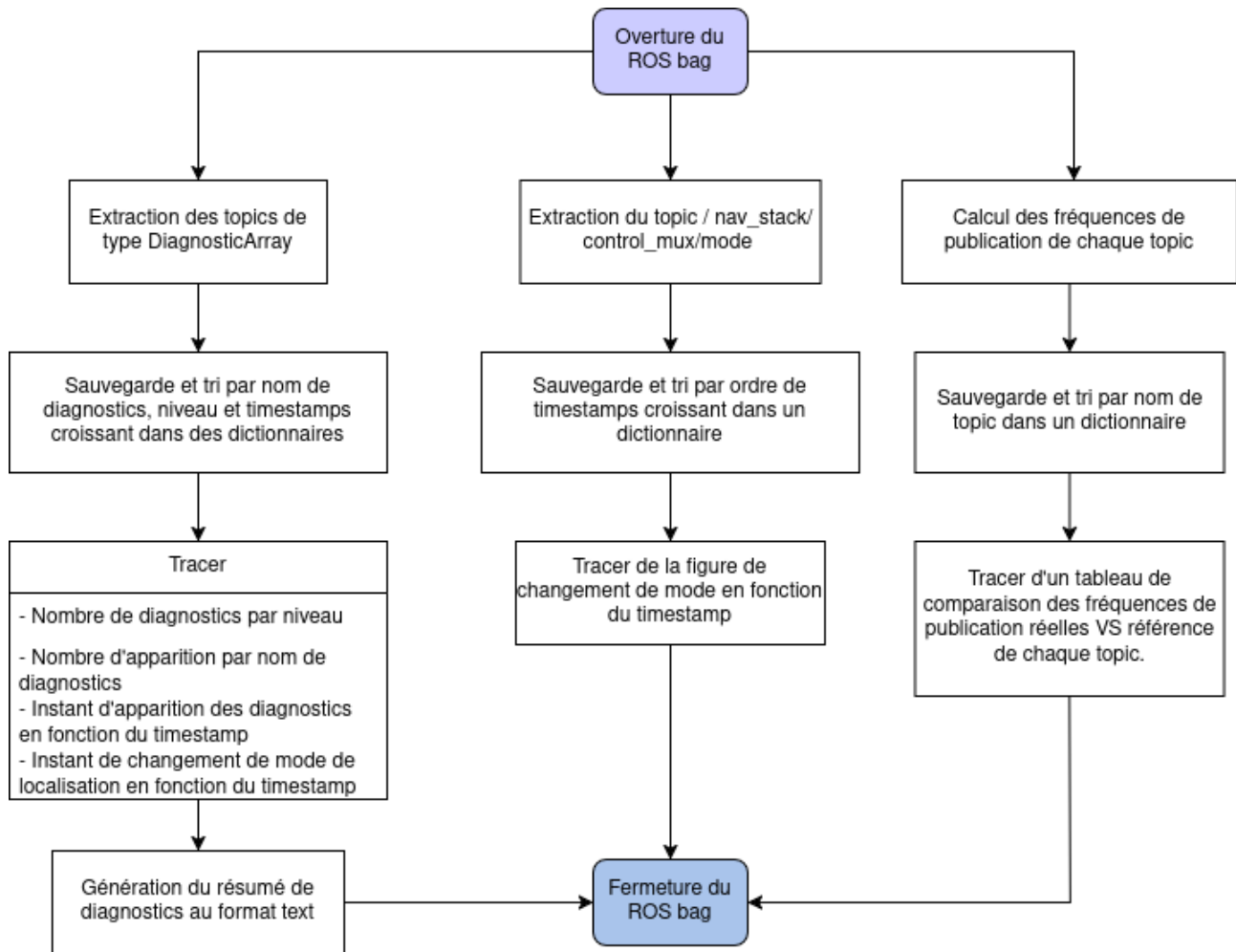


FIGURE 34 – Organigramme de l'outil d'extraction et de génération des diagnostics

2.4.2 Extraction et génération des visualisation de trajectoires et d'erreurs de localisation

Ce module a pour objectif principal de fournir une représentation visuelle et quantitative des trajectoires du robot (estimées par le SLAM et la centrale inertielle SBG Ellipse) au regard des waypoints (points de passage) à rallier. Afin d'évaluer et de comparer objectivement ces profils de déplacement, la bibliothèque Python EVO (Evaluation of Odometry and SLAM) a été intégrée à l'architecture de test.

L'outil automatisé exploite cette bibliothèque pour générer plusieurs types de livrables graphiques :

- **Des analyses temporelles** : l'évolution des positions linéaires et angulaires du robot au fil du temps.(Figure 34)
- **Des comparaisons spatiales** : la superposition des trajectoires issues du SLAM et de la centrale inertielle SBG.(Figure 35)
- **Des métriques d'erreur** : le calcul et l'affichage de l'erreur de position relative (RPE — Relative Pose Error) pour quantifier la dérive locale du système.

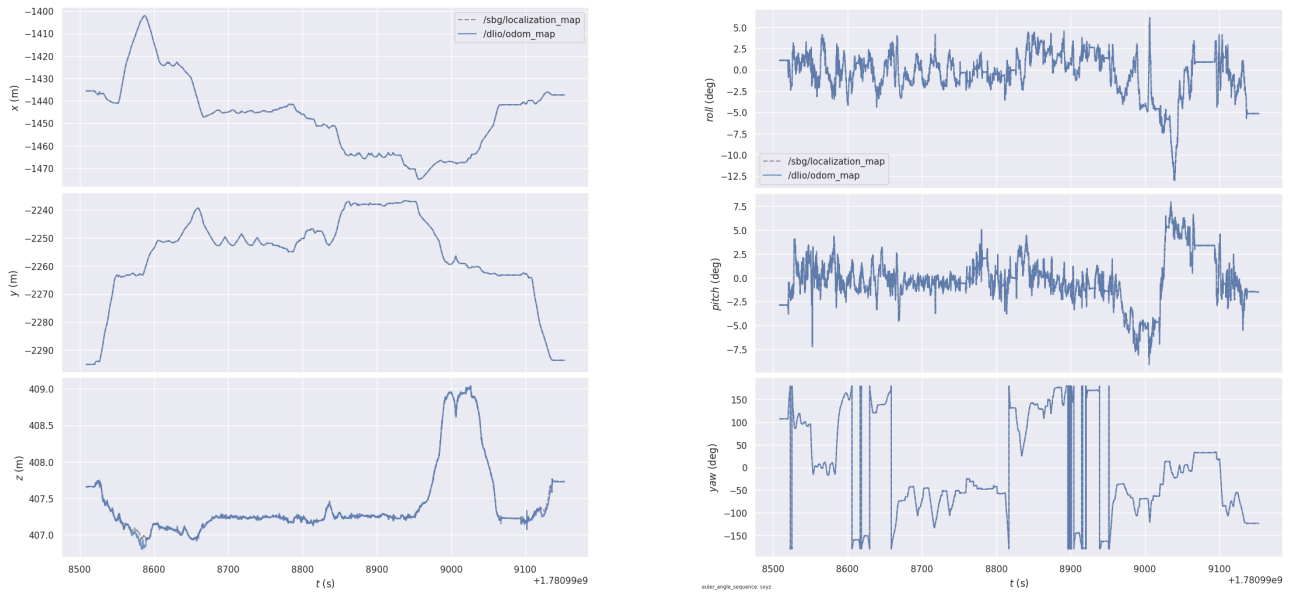


FIGURE 35 – Visualisation et comparaison des positions linéaires et angulaires du robot

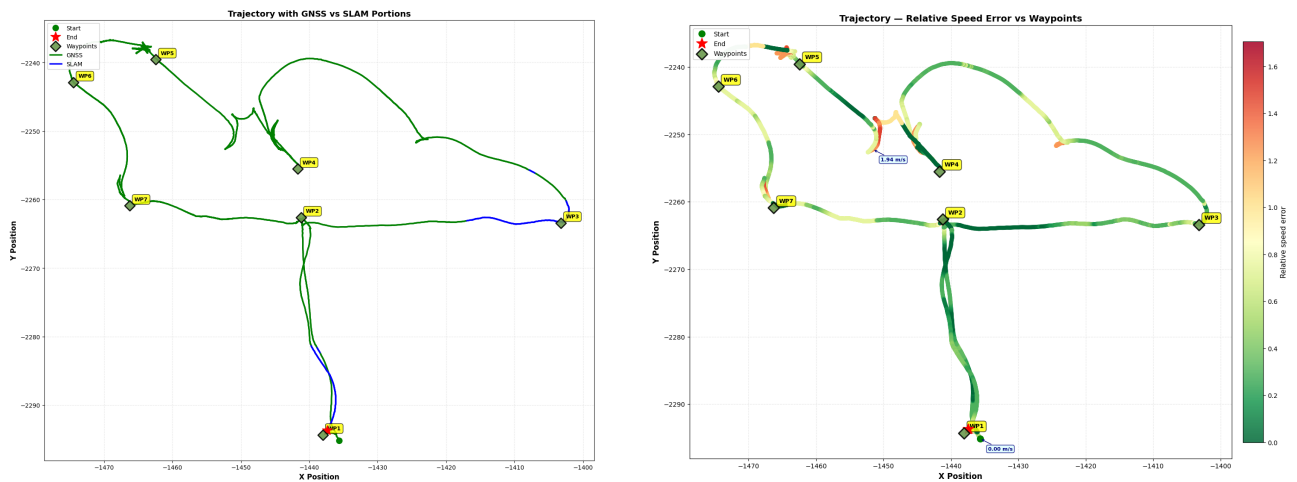


FIGURE 36 – Evaluation des portions GNSS et SLAM de la trajectoire et erreur de vitesse le long de la trajectoire

Ces visualisations synthétiques permettent d'identifier rapidement les écarts géométriques entre les différentes méthodes de localisation et d'analyser finement les performances de la brique de navigation selon les scénarios de test. Il convient toutefois de souligner que la pertinence de ces indicateurs de performance (KPI) reste conditionnée à la validité de la vérité terrain (ground truth). Celle-ci nécessite une continuité absolue des données de positionnement, ce qui implique l'absence de masquage ou de perte du signal GNSS RTK durant l'acquisition des données.

Afin d'optimiser ce traitement, l'architecture de cet outil a été subdivisée en deux modules distincts :

- Le module d'extraction et de sérialisation : Cette première partie est chargée d'extraire les informations pertinentes des topics du fichier bag, puis de les enregistrer dans un fichier CSV. Les données y sont structurées sous forme de tableau indexé par horodatage (timestamp), où chaque colonne correspond à un attribut spécifique et exploitable du message d'origine.
- Le module d'analyse et de visualisation : Cette seconde partie assure la lecture du fichier CSV et prend en charge la génération des différentes figures d'analyse.

Cette conception modulaire et découplée présente un double intérêt opérationnel. D'une part, elle évite de réexécuter la phase d'extraction des données du bag (processus généralement lourd et chronophage) lors d'un simple ajustement graphique ou d'une modification des figures d'analyse. D'autre part, elle permet de générer un fichier CSV intermédiaire. Ce support, considérablement plus léger qu'un fichier bag, assure la persistance de données pré-traitées et facilement exploitables pour des analyses ultérieures.

Pour la première partie du traitement, l'outil génère ainsi un fichier CSV indexé par horodatage contenant les colonnes associées aux différents champs des messages pour les topics de type :

- **geometry_msgs/TwistStamped** : Champ linéaire et angulaire de la vitesse afin d'avoir les vitesses de consigne envoyées au robot à chaque instant
- **nav_msgs/Odometry** : Pose du robot dans les différents modes de localisation (SLAM, SBG) pour des tracés de trajectoires.
- **nav_msgs/Path** : Liste et coordonnées des waypoints à rallier par le robot pour les tracés sur trajectoires.
- **sensor_msgs/Imu** : Mesures inertielle pour les tracés d'orientation du robot tout au long de la trajectoire.

Pour la seconde partie du traitement, l'outil exploite les données du CSV pour générer une série de visualisations détaillées, complétée par des métriques d'évaluation standardisées. Ces livrables graphiques se divisent en deux catégories principales :

a. Les figures d'analyse dynamique et comportementale :

Ces tracés permettent de caractériser finement le comportement cinématique et l'état du système tout au long de son parcours :

- **Analyse spatiale et trajectographie** : Superposition et comparaison des trajectoires estimées par le SLAM et par la centrale inertielle (SBG Systems), avec projection des points de passage (waypoints) cibles.
- **Modes de localisation** : Visualisation temporelle et géographique du mode de localisation actif (SLAM ou GNSS RTK) sur les différentes portions de la trajectoire.
- **Attitude du châssis** : Évolution des angles d'orientation de roulis (roll) et de tangage (pitch) du robot le long du tracé, permettant d'identifier d'éventuelles perturbations liées au relief.
- **Suivi de consigne en vitesse** : Confrontation des vitesses linéaires et angulaires de consigne avec les vitesses réelles mesurées sur le robot.
- **Analyse des écarts de vitesse** : Évaluation des erreurs de vitesse instantanées par

rapport aux vitesses de consigne requises pour le ralliement des waypoints.

- **Performances temporelles du suivi** : Mesure fine de la durée de ralliement pour chaque waypoint intermédiaire, ainsi que le calcul du temps total d'exécution de la mission de suivi de trajectoire.

b. Les figures d'évaluation géométrique de la trajectoire :

En complément de ces analyses comportementales, l'outil intègre un module d'évaluation métrologique s'appuyant sur la bibliothèque open-source EVO. Ce module génère automatiquement des figures d'évaluation standardisées, permettant notamment de quantifier l'erreur de pose absolue (APE - Absolute Pose Error) et l'erreur relative de pose (RPE - Relative Pose Error) par rapport à la trajectoire de référence comme présenté dans la section 2.3.

Le processus débute par l'analyse du fichier d'enregistrement (bag file), dont l'objectif est d'extraire les topics de localisation SLAM (`/dlio/odom`) et SBG (`/sbg/localization`) ainsi que les waypoints à rallier (`/nav_stack/function_waypoints/waypoints_iterator/path_in`), mais aussi les commandes en vitesses envoyées au robot afin de les comparer à celle suivies par le robot à chaque instant, . Une fois l'extraction effectuée, les informations utiles sont structurées et stockées chronologiquement au sein d'un dictionnaire indexé par horodatage (timestamp). À partir de cette structure de données, des scripts exploitant des bibliothèques Python (pandas, numpy, matplotlib) génèrent automatiquement l'ensemble des figures d'analyse présentées précédemment. Enfin, afin de mettre en relation les anomalies matérielles ou logicielles avec l'état opérationnel du robot, l'outil extrait également les messages du topic `nav_stack/control_mux/mode`. L'analyse de ces données permet de tracer et de corréler précisément les changements de modes de contrôle du système au cours de l'essai. La figure (Figure 36) suivante illustre l'organigramme de cet outil :

2.4.3 Extraction et génération des vidéos de l'évolution des cartes et du retour vidéo sur l'IHM

Ce module a pour objectif principal de fournir une représentation visuelle dynamique (vidéo) de l'évolution du MNE (Modèle Numérique d'Élévation), de la planification globale et locale, ainsi que du retour vidéo sur l'interface homme-machine (IHM). L'outil automatise l'extraction des images de chaque topic images contenues dans le bag puis génère des vidéos associées. Cette vidéo permet d'observer en temps réel la construction de la carte, la planification des trajectoires et la réponse du robot aux commandes, offrant ainsi un aperçu global du comportement du système durant les tests.

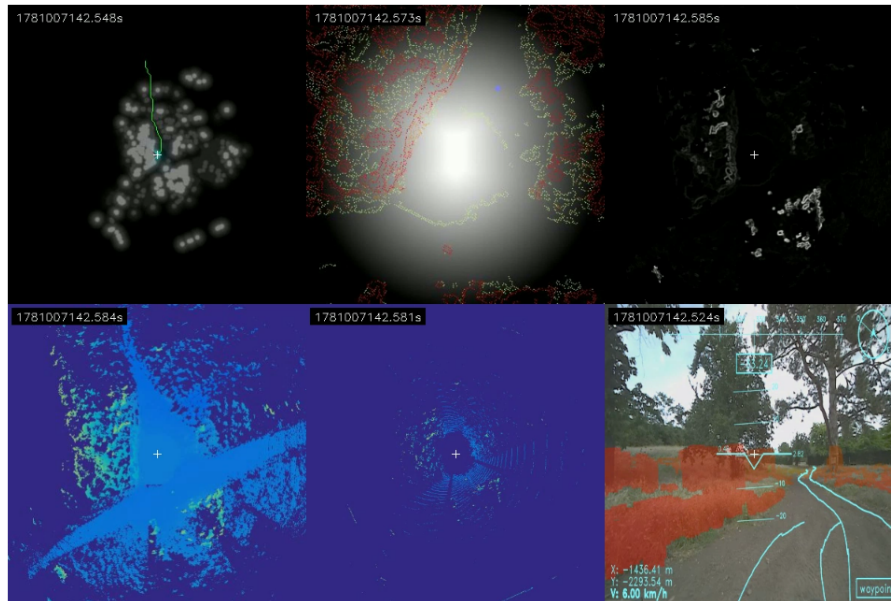


FIGURE 37 – Vidéo composite des différentes cartes et du retour vidéo sur l'IHM

2.4.4 Structuration modulaire du générateur de rapports

Une fois le générateur de rapports de test rendu fonctionnel, l'objectif suivant a consisté à lui apporter une structure modulaire. Cette modularité devait permettre à l'utilisateur de sélectionner à la carte les informations à intégrer dans le rapport final, parmi les options suivantes : diagnostics, visualisation de trajectoires et vidéos. Pour ce faire, j'ai suivi les recommandations d'un ingénieur de l'équipe et me suis tourné vers les bibliothèques Rich et Typer. Rich permet d'améliorer l'affichage dans le terminal, rendant l'expérience utilisateur plus attrayante, tandis que Typer facilite la création d'applications en ligne de commande avec la possibilité de passer des arguments pour spécifier les éléments à générer dans le rapport. L'outil se présente comme le montre la figure suivante (Figure 37) :

```
sherpa-moi@mobiter:~/mobilexworkspace/mobitervalidationprocedure$ python3 Analyser_CLI.py --help

Usage: Analyser_CLI.py [OPTIONS] COMMAND [ARGS]...

Modular extraction & analysis for ROS bag files.

Options
--help          Show this message and exit.

Commands
info           Show topics and message-type statistics stored in a bag file.
diag          Extract and visualize diagnostic messages from a bag file.
video         Extract videos from Image / CompressedImage topics in a bag file.
visualize     Extract topic data then visualize - topics + plots in a single
              command.
all           Run all extractions in sequence: diagnostics → video → topics →
              visualize.
record        Start live rosbag recording of TF-transformed topics.
```

FIGURE 38 – Visualisation terminal de l'outil de génération automatique de rapport de test

2.5 Mise en place du plan de test

2.6 Proposition de méthode d'automatisation de la validation de tests en robotique mobile : cas d'application sur Mobiter

2.7 Robustification du mode suivi de lisière : gestion d'obstacles sur le chemin de suivi

Comme présenté dans la section 2.2, le mode de suivi de lisière est un mode de navigation autonome conçu pour permettre au robot de longer une lisière d'arbres ou de végétation. Cependant, lors des campagnes de tests, ce mode a révélé des limites opérationnelles majeures en présence d'obstacles non franchissables obstruant le chemin de suivi.

En effet, face à un obstacle, le robot s'immobilisait systématiquement. Ce blocage s'explique par le paramétrage de sécurité du planificateur local (local planner) associé à ce mode, configuré pour inhiber les commandes de vitesse générées par le correcteur de pure poursuite (pure pursuit) dès qu'une trajectoire entrerait en collision avec un obstacle. Ce comportement passif s'est avéré particulièrement pénalisant et non viable dans le cadre du Défi 3 du challenge.

Pour pallier cette faiblesse, une première approche a consisté à optimiser l'algorithme de détection de lisière en élargissant le rayon du cercle d'analyse. L'objectif était d'intégrer les obstacles proches de la lisière (seuil fixé à 1,5 mètre) afin qu'ils soient directement assimilés à la lisière elle-même, forçant ainsi le robot à la contourner naturellement. Néanmoins, cette solution s'est avérée insuffisante pour deux raisons majeures :

- **Obstacles isolés** : Elle ne permettait pas de traiter les obstacles situés au centre du chemin de suivi, trop éloignés de la lisière pour entrer dans le rayon d'analyse étendu.
- **Perte de référence visuelle (Masquage)** : Lors de l'amorce de la manœuvre d'évitement, la trajectoire d'évitement éloignait le robot de sa trajectoire nominale, lui faisant perdre de vue la lisière de référence qu'il suivait initialement.

De plus, l'augmentation arbitraire du rayon du cercle de détection a introduit un effet de bord critique. Cette modification réduit en effet l'aptitude du système à détecter la continuité de la lisière située immédiatement derrière l'obstacle (si celui-ci n'est pas très proche de la lisière), créant ainsi une "zone d'ombre" algorithmique, comme l'illustre la figure ci-dessous (Figure 38) :

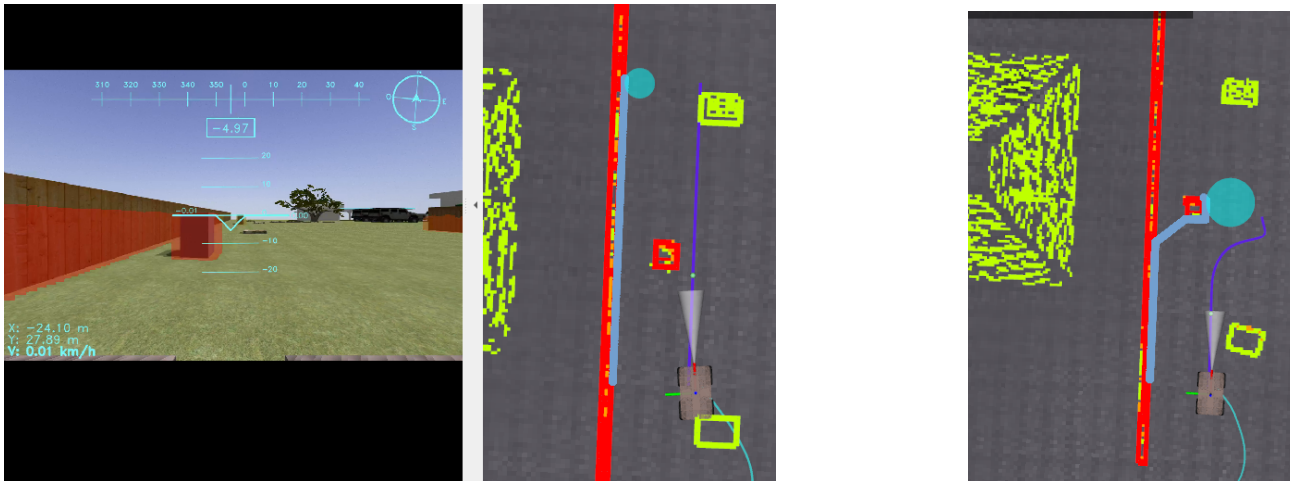


FIGURE 39 – Evaluation de la taille du cercle de détection de lisière sur la capacité à détecter la continuité de la lisière derrière un obstacle

L'idée suivante était de garder un petit cercle et d'introduire un mécanisme de mise à jour intelligent du chemin de suivi. Au début de mon stage, ce chemin était actualisé à chaque nouvelle détection de lisière, ce qui posait des problèmes lors des manœuvres de contournement d'obstacle car un nouveau chemin était calculé à partir de la lisière détectée sur l'obstacle lors de la manœuvre. L'objectif était donc de mettre à jour le chemin courant si et seulement si le nouveau chemin issu de la nouvelle détection de lisière est cohérent. Par cohérent, nous entendons un test de longueur de chemin car la longueur d'une lisière suivie ne baisse pas drastiquement entre deux déplacements élémentaires successifs du robot.

Face aux limites des ajustements purement géométriques, l'approche retenue a consisté à conserver un rayon de détection restreint (garantissant un bon pouvoir de résolution) tout en introduisant un mécanisme de filtrage et de mise à jour dynamique cohérent du chemin de suivi. Initialement, le chemin de référence du robot était recalculé et écrasé de manière asynchrone à chaque nouvelle détection de lisière. Lors des phases de contournement, ce comportement générerait des instabilités critiques : l'algorithme interprétait l'obstacle lui-même comme la nouvelle lisière à suivre, perturbant ainsi la trajectoire globale du robot.

Pour résoudre ce problème, une logique de transition d'état conditionnelle a été implémentée. Désormais, le chemin courant n'est mis à jour par une nouvelle détection que si cette dernière respecte un critère de cohérence spatio-temporelle. Ce filtre repose sur l'hypothèse physique selon laquelle la longueur d'une lisière continue ne peut subir de variation drastique (diminution brutale) entre deux pas de calcul successifs du système (de l'ordre de quelques millisecondes). Mathématiquement, soit L_t la longueur du chemin issu de la détection à l'instant t , et L_{t-1} la longueur du chemin précédemment validé. La mise à jour du chemin de navigation est autorisée si et seulement si la longueur du nouveau chemin respecte l'équation suivante (??) :

$$\frac{L_{t-1}}{2} \leq L_t \quad (8)$$

- En cas de détection cohérente : Si la condition est respectée, le chemin est mis à jour, garantissant l'adaptation du robot aux légères courbures de la lisière.
- En cas d'anomalie ou de masquage : Si la condition n'est pas respectée, la mise à jour

est inhibée. Le robot conserve et suit son chemin précédent, ce qui lui permet de franchir la zone de perturbation (l'obstacle).

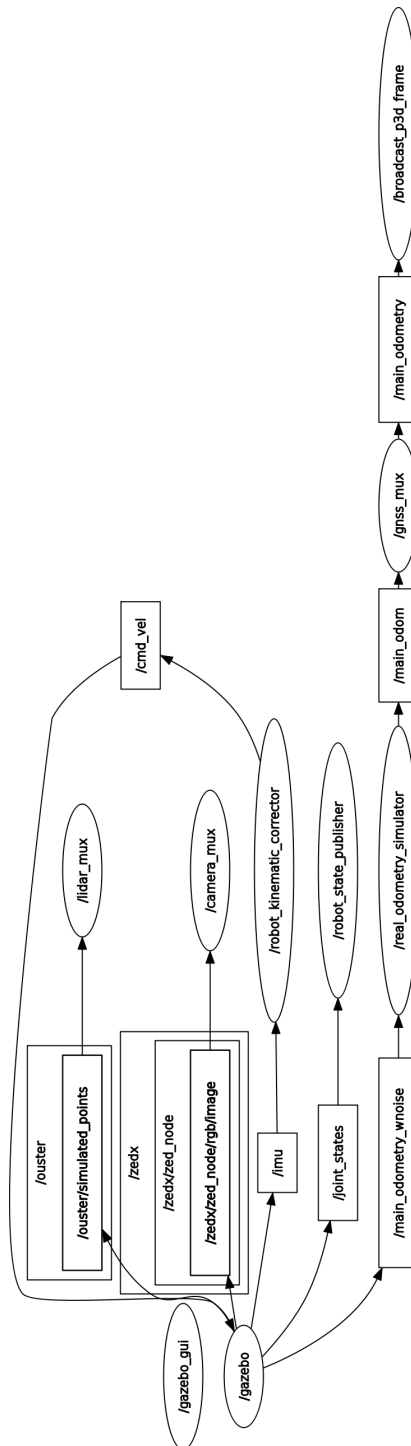
Table des matières

Annexes

Table des annexes

Annexe 1 : RQT Graph du simulateur Gazebo	??
Annexe 2 : Checklist du rapport	??

Annexe 1 : RQT graph du simulateur Gazebo



Annexe 2 : Checklist du rapport

Couverture		logos, titre, prénoms/noms des étudiants, noms des tuteurs, mention « rapport de stage » ou « rapport de projet », année, département ; nom de l'école en entier (Polytech Clermont-Ferrand)
Résumés, mots clés		Résumé + mots clés Abstract / key words
Remerciements		ordre (en gras ce qui est obligatoire) : tuteur entreprise , personnels entreprise, tuteur école , enseignants école, autres « Mr. » = Mister ; Monsieur = « M. » Attention à l'accord des participes passés
Sommaire		2 niveaux de titres
Table des figures et des tableaux		toutes les figures et tous les tableaux sont répertoriés classement dans l'ordre d'apparition des figures et tableaux « Figure 1 : Titre, numéro de page »
Glossaire		- les mots du domaine - présentés en ordre alphabétique - en fin de lexique : une mention indiquant que « tous les mots suivis d'un astérisque sont définis dans le lexique » - dans le texte : un astérisque à chaque mot défini dans le lexique
Table des abréviations		- obligatoire s'il y a utilisation d'abréviations - ordre alphabétique
Introduction		Accroche, sujet, problématique, entreprise, enjeux, méthode de travail, annonce du plan
Conclusion		rappel de la problématique, rappel synthétique des résultats, distance critique par rapport à ces résultats, ouverture vers un autre sujet ou une autre problématique
Bilan		prise de recul, analyse des compétences acquises
Bibliographie sitographie		Application des consignes ATTENTION à bien citer dans le texte toutes les références
Table des matières		tous les niveaux de titres
Figures		Elles sont TOUTES : numérotées, titrées, référencées dans le texte, sourcées
Annexes		table des annexes (« Annexe 1 : TITRE, page ») les annexes sont numérotées et classées en ordre d'apparition dans le texte, elles sont toutes appelées dans le texte, paginées en chiffres romains, à partir de 1